CPU Pipelining

CS2600 (3003)

By

Dr. Sk Subidh Ali

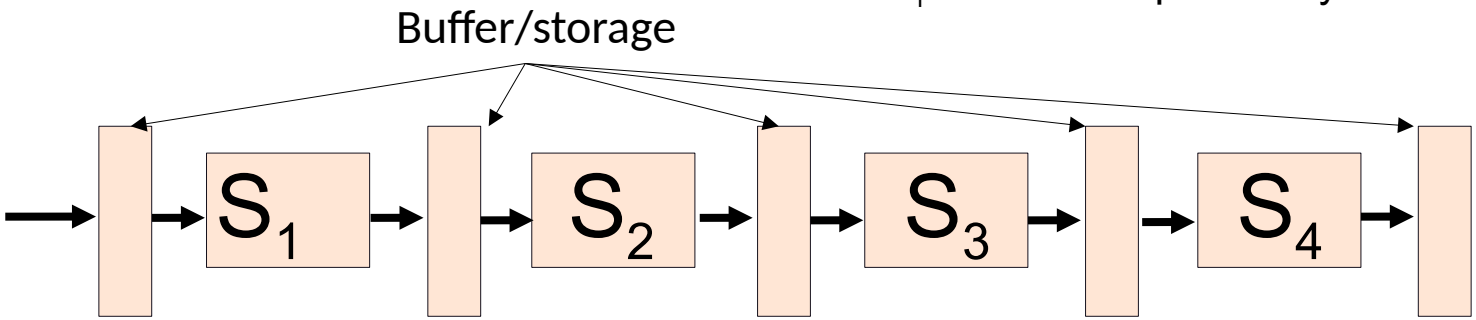
CPU Task Execution

T: Task, i.e., an executable to be executed in a CPU

$T^j = T^j_1 + T^j_2 + T^j_3 + T^j_4$

T^j: Can be divided into four subtask

A subtask T^j_i can be independently executed in block S_i



Block B_i can execute T_i in one cycle

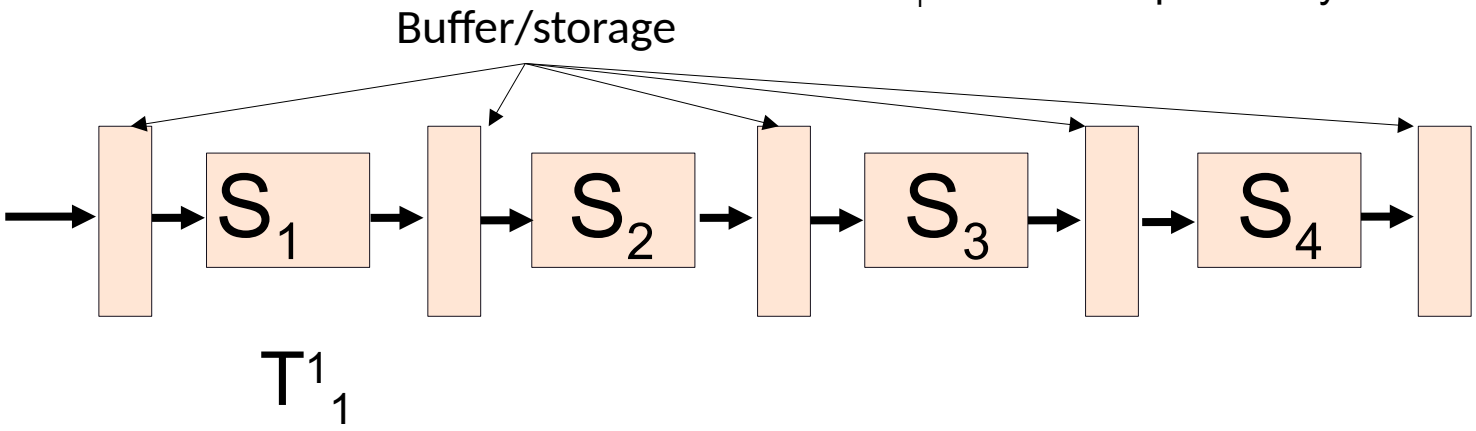
CPU Task Execution

T: Task, i.e., an executable to be executed in a CPU

$T^j = T^j_1 + T^j_2 + T^j_3 + T^j_4$

T^j: Can be divided into four subtask

A subtask T^j_i can be independently executed in block S_i



CLK_1

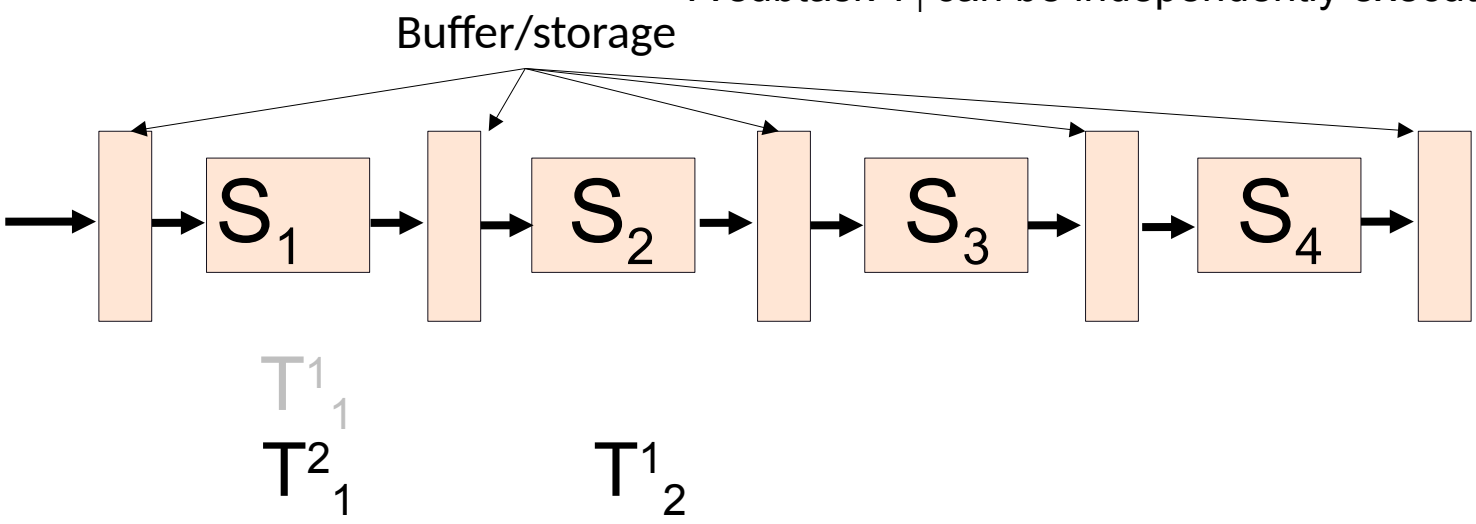
CPU Task Execution

T: Task, i.e., an executable to be executed in a CPU

$T^j = T^j_1 + T^j_2 + T^j_3 + T^j_4$

T^j: Can be divided into four subtask

A subtask T^j_i can be independently executed in block S_i



CLK_2

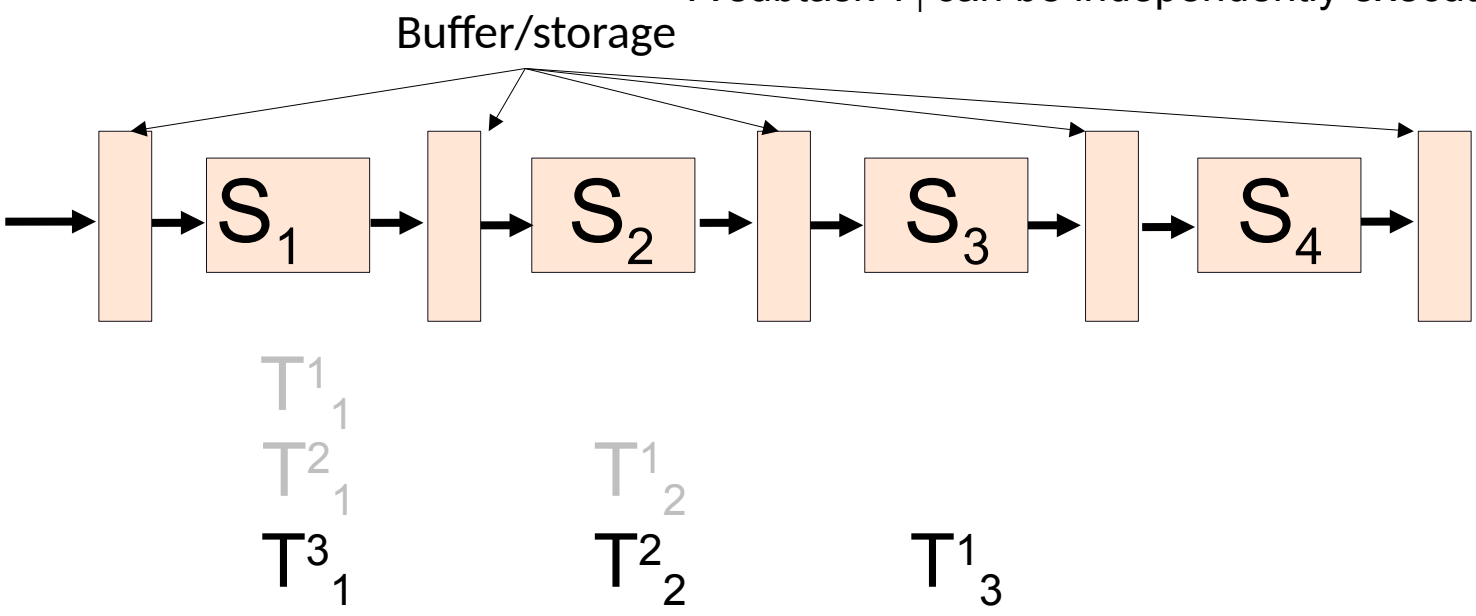
CPU Task Execution

T: Task, i.e., an executable to be executed in a CPU

$T^j = T^j_1 + T^j_2 + T^j_3 + T^j_4$

T^j: Can be divided into four subtask

A subtask T^j_i can be independently executed in block S_i



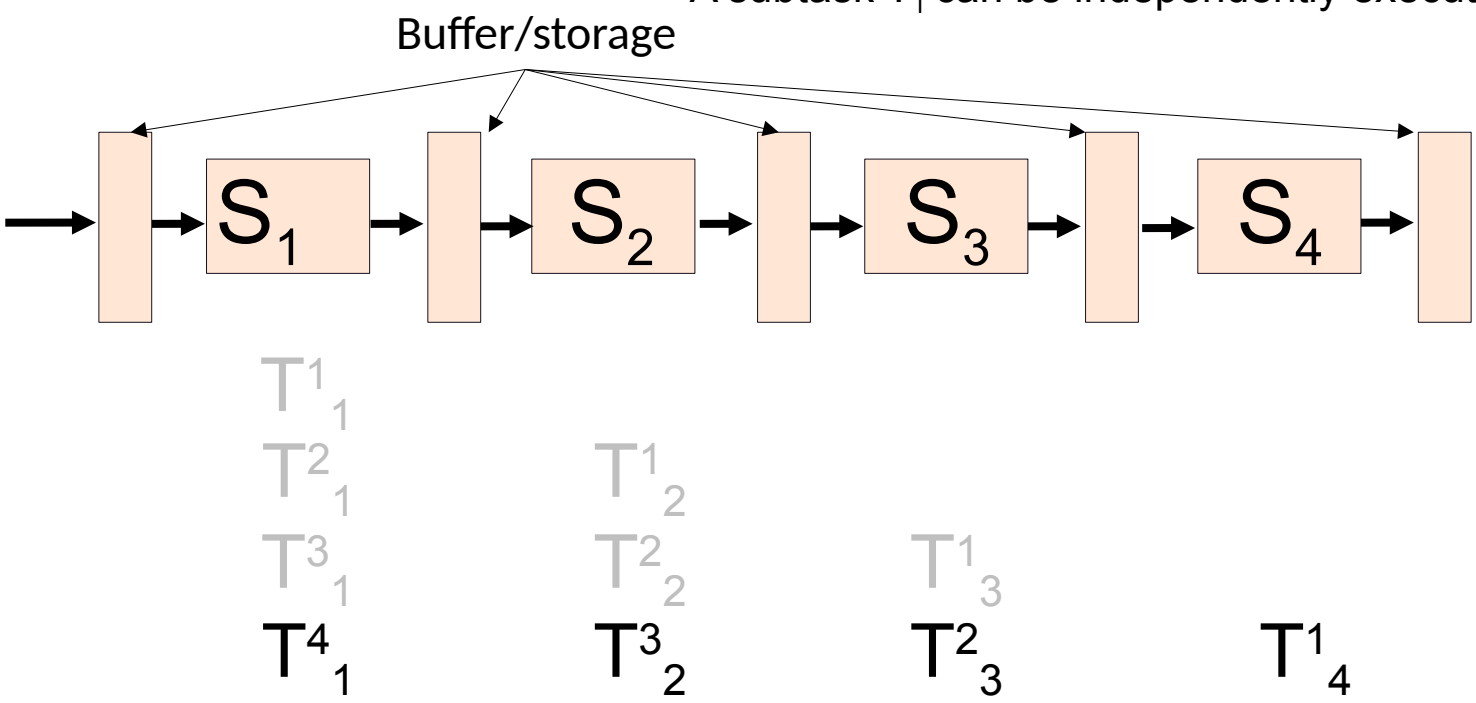
CPU Task Execution

T: Task, i.e., an executable to be executed in a CPU

$T^j = T^j_1 + T^j_2 + T^j_3 + T^j_4$

T^j: Can be divided into four subtask

A subtask T^j_i can be independently executed in block S_i



The pipeline is full: each stage is busy

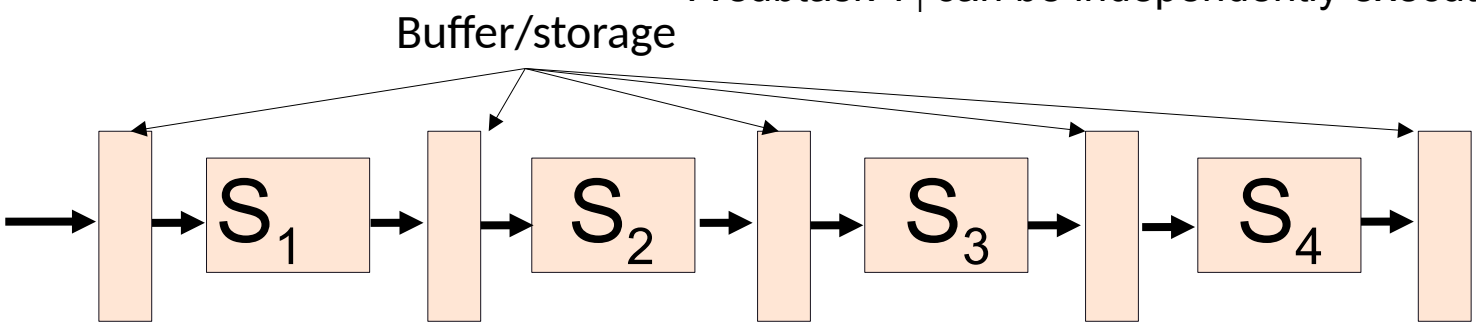
CPU Task Execution

T: Task, i.e., an executable to be executed in a CPU

$T^j = T^j_1 + T^j_2 + T^j_3 + T^j_4$

T^j: Can be divided into four subtask

A subtask T^j_i can be independently executed in block S_i



T^5_1 T^4_2 T^3_3 T^2_4

CLK_5

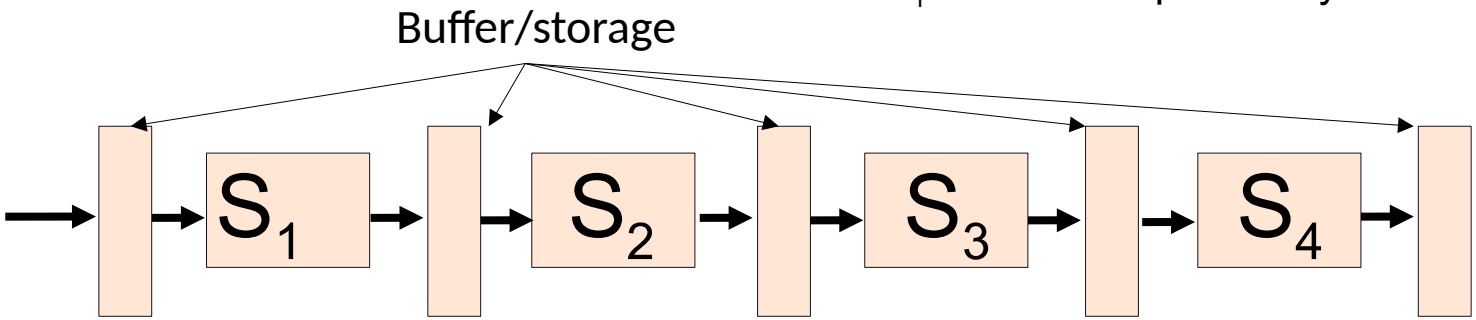
CPU Task Execution

T: Task, i.e., an executable to be executed in a CPU

$T^j = T^j_1 + T^j_2 + T^j_3 + T^j_4$

T^j: Can be divided into four subtask

A subtask T^j_i can be independently executed in block S_i



How many cycles required to complete n tasks?

T^6_1 T^5_2 T^4_3 T^3_4

CLK_6



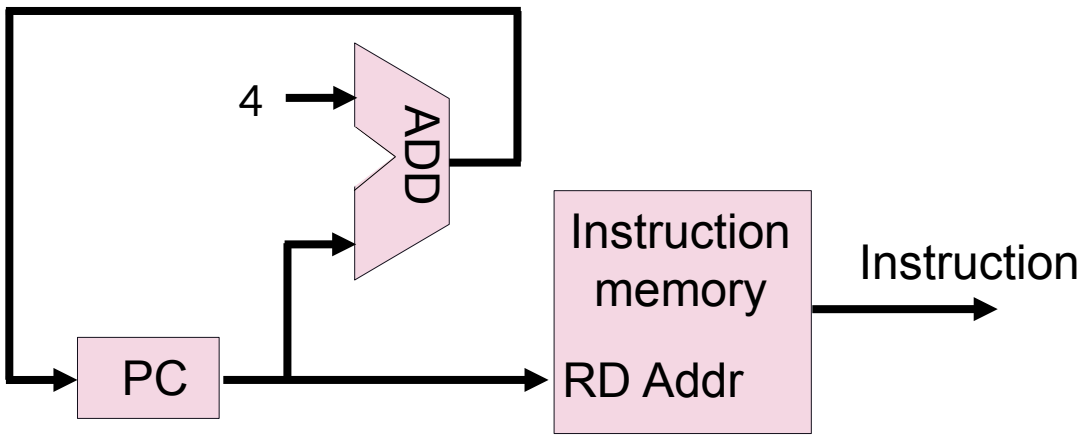
CPU Instruction Pipelining

Case studies:

- Register referencing: add R_1, R_2, R_3 => R_1=R_2 + R_3
- Memory referencing: lb R_1, imm(R_2) => R_1= [R_2+ imm] // n is a constant number
  sb R_1, imm(R_2) => [R_2+ imm] = R_1 // n is a constant number
- Jump beq R_1, R_2, L => Jump to L if R_1=R_2 // L is offset value

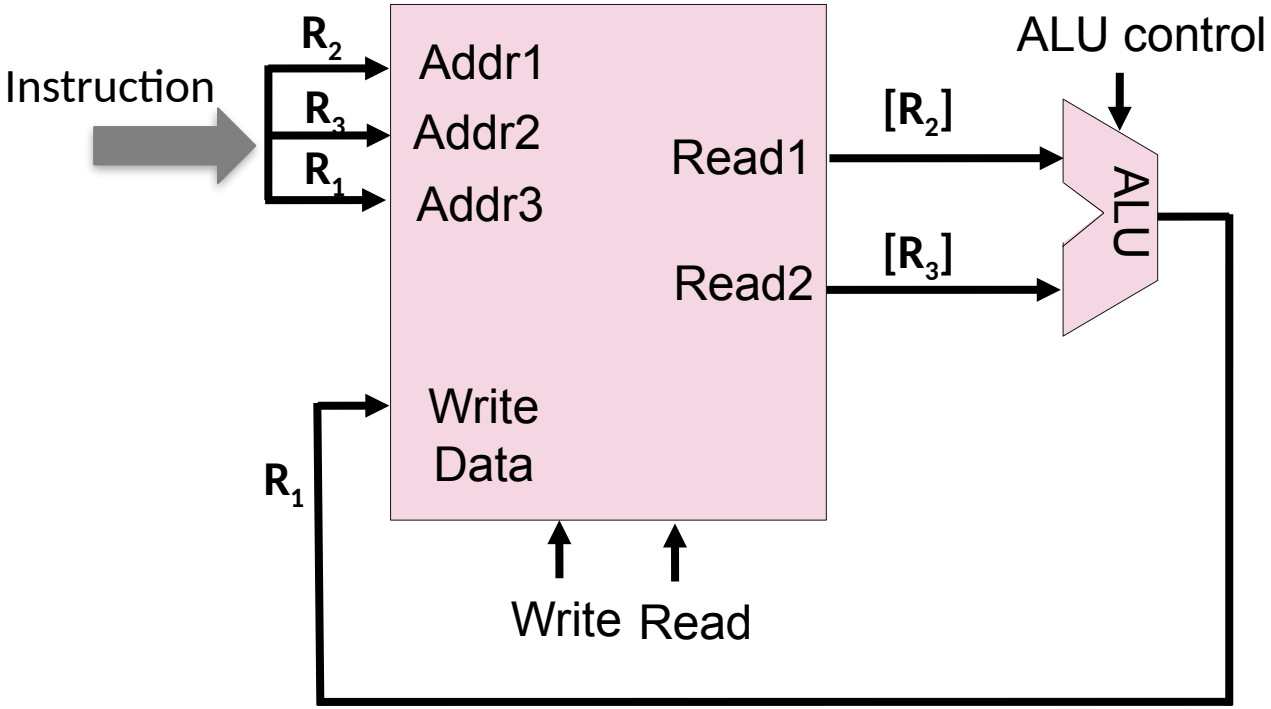
Consider each instruction is of 4 bytes or 32 bits

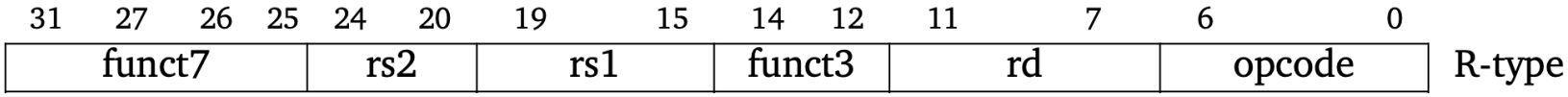
Instruction fetch (IF)



CPU Instruction Pipelining

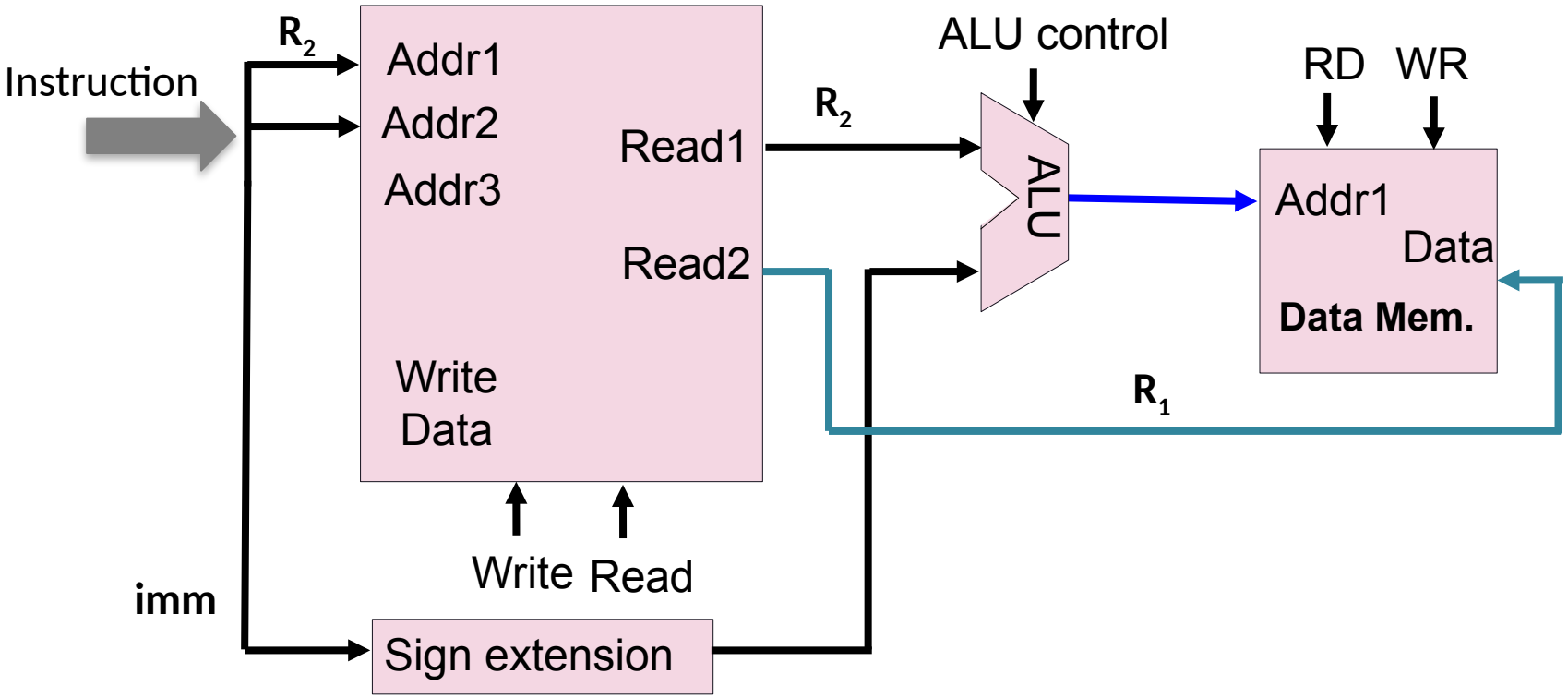
- Register referencing: **add R_1, R_2, R_3**





CPU Instruction Pipelining

- Memory referencing: lb R_1, imm(R_2) => R_1= [R_2+ imm] // imm is a constant number

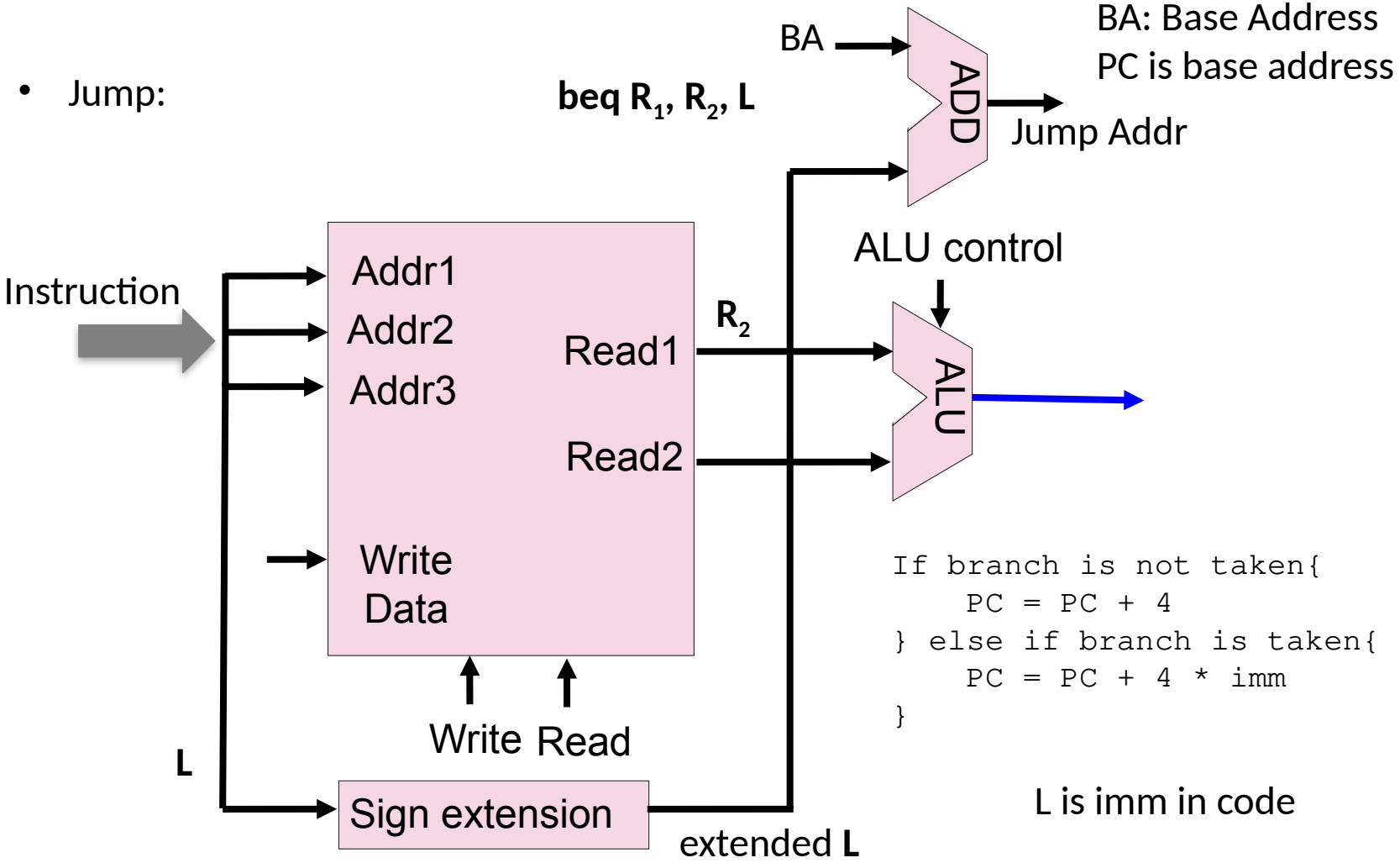


31 27	26 25	24 20	19 15	14 12	11 7	6 0	
imm[11:0]			rs1	funct3	rd	opcode	I-type
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode	S-type

CPU Instruction Pipelining

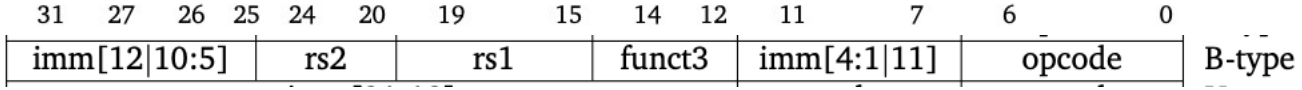
- Jump:

beq R_1, R_2, L



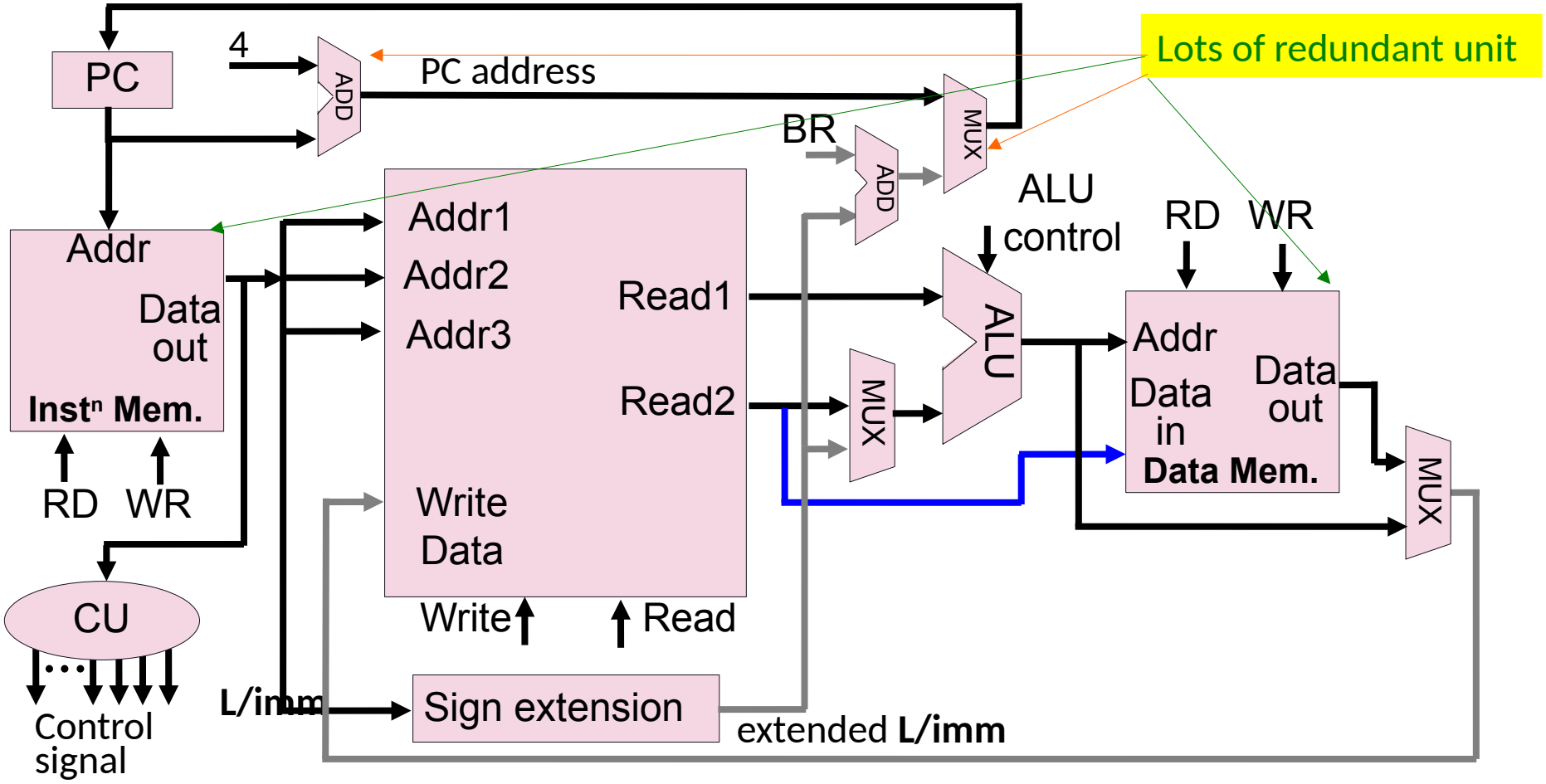

```
If branch is not taken{
    PC = PC + 4
} else if branch is taken{
    PC = PC + 4 * imm
}
```

L is imm in code

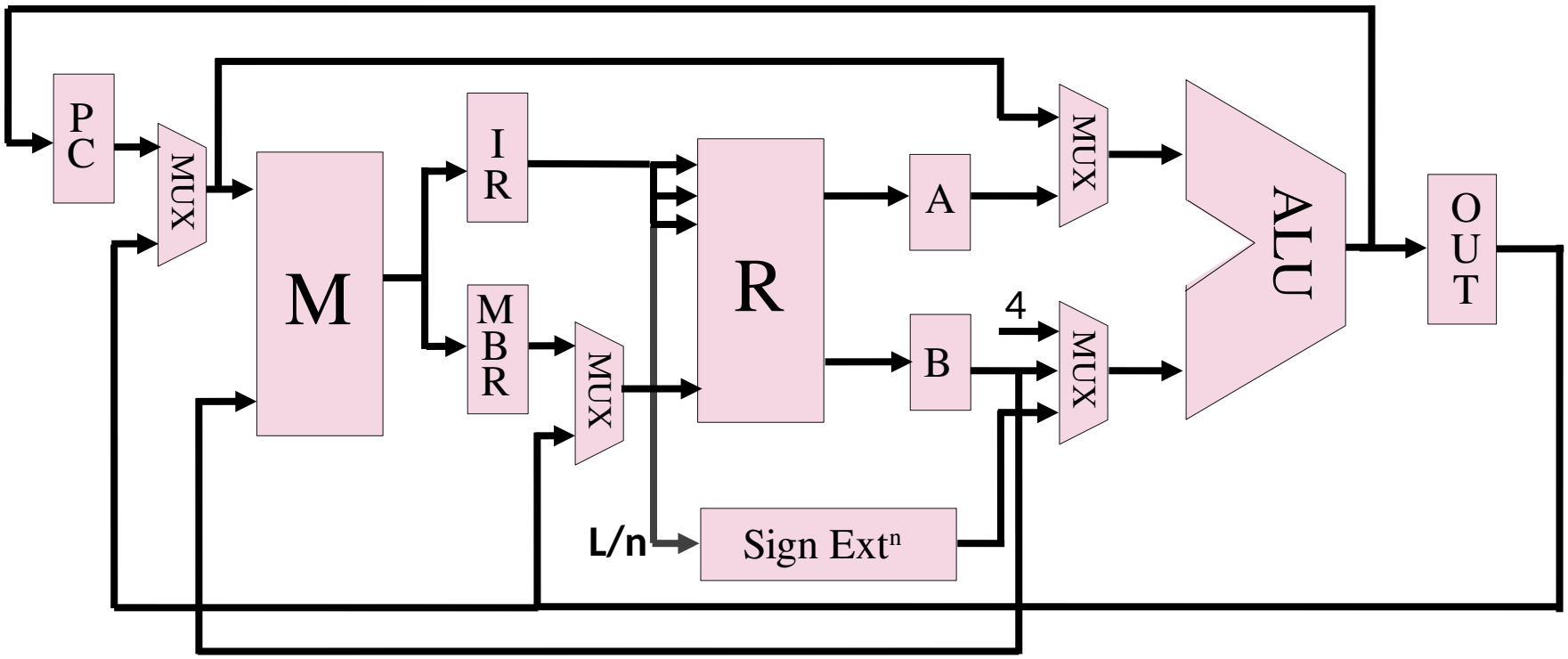


CPU Design

add R_1, R_2, R_3 **sb R_1, imm(R_2)** **lb R_1, n(R_2)** **beq R_1, R_2, L**



Optimized Design of CPU



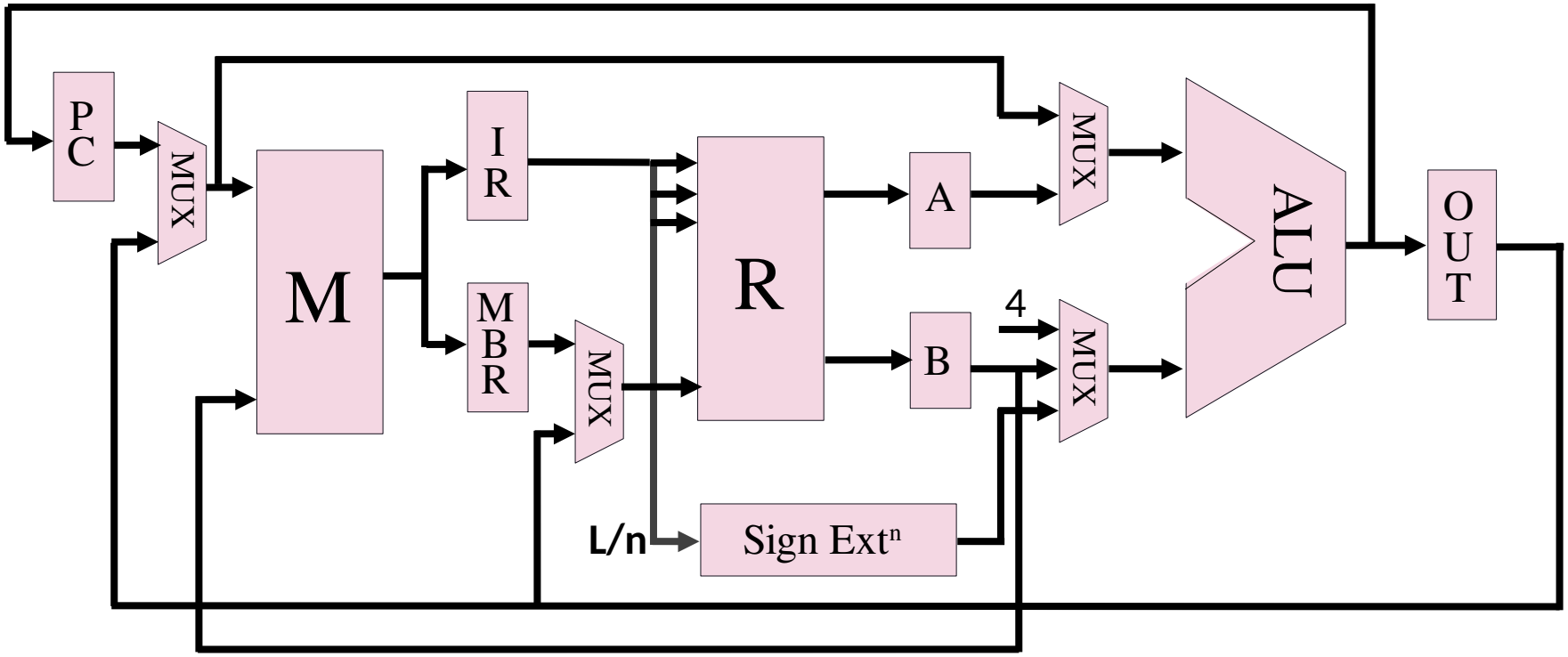
Optimized Design of CPU

Instruction execution

Fetch

IR=M[PC]

PC=PC+4



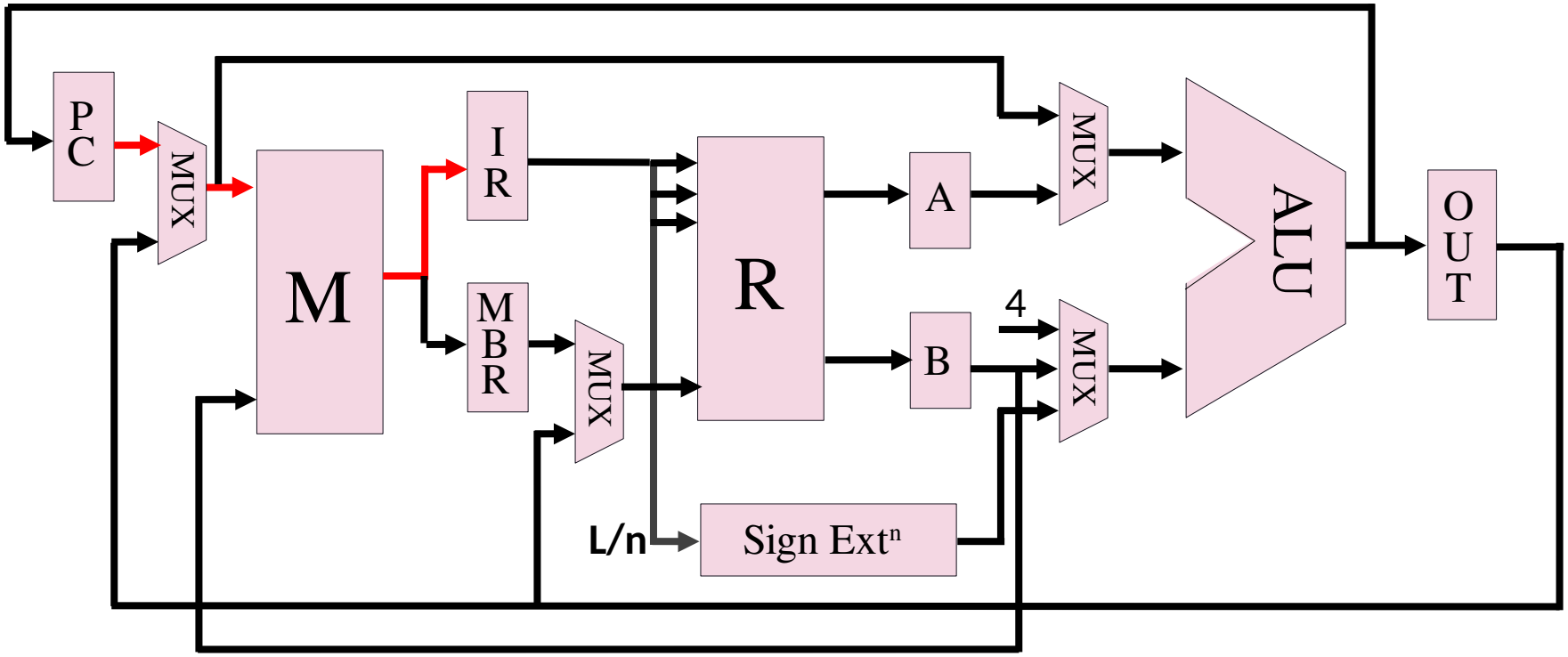
Optimized Design of CPU

Instruction execution

Fetch

IR=M[PC]

PC=PC+4



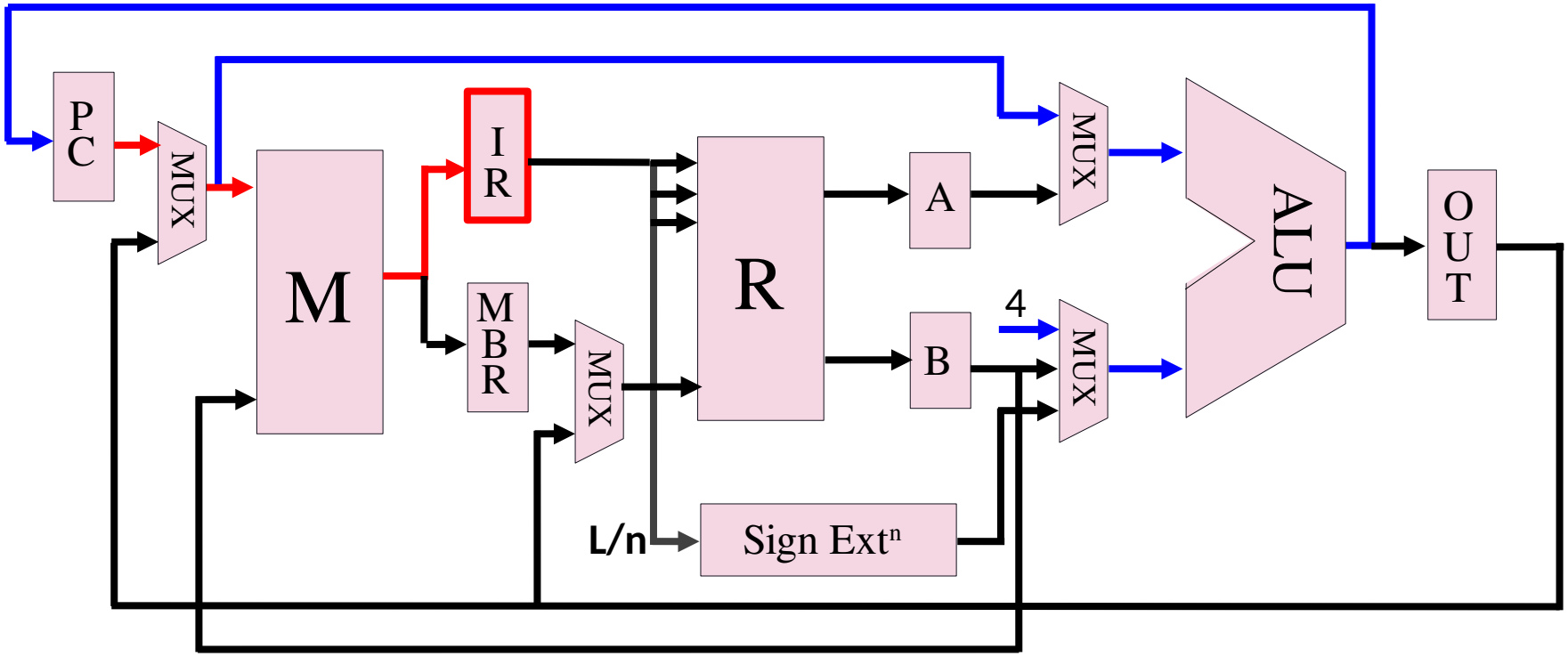
Optimized Design of CPU

Instruction execution

T_0 Fetch

IR=M[PC]

PC=PC+4

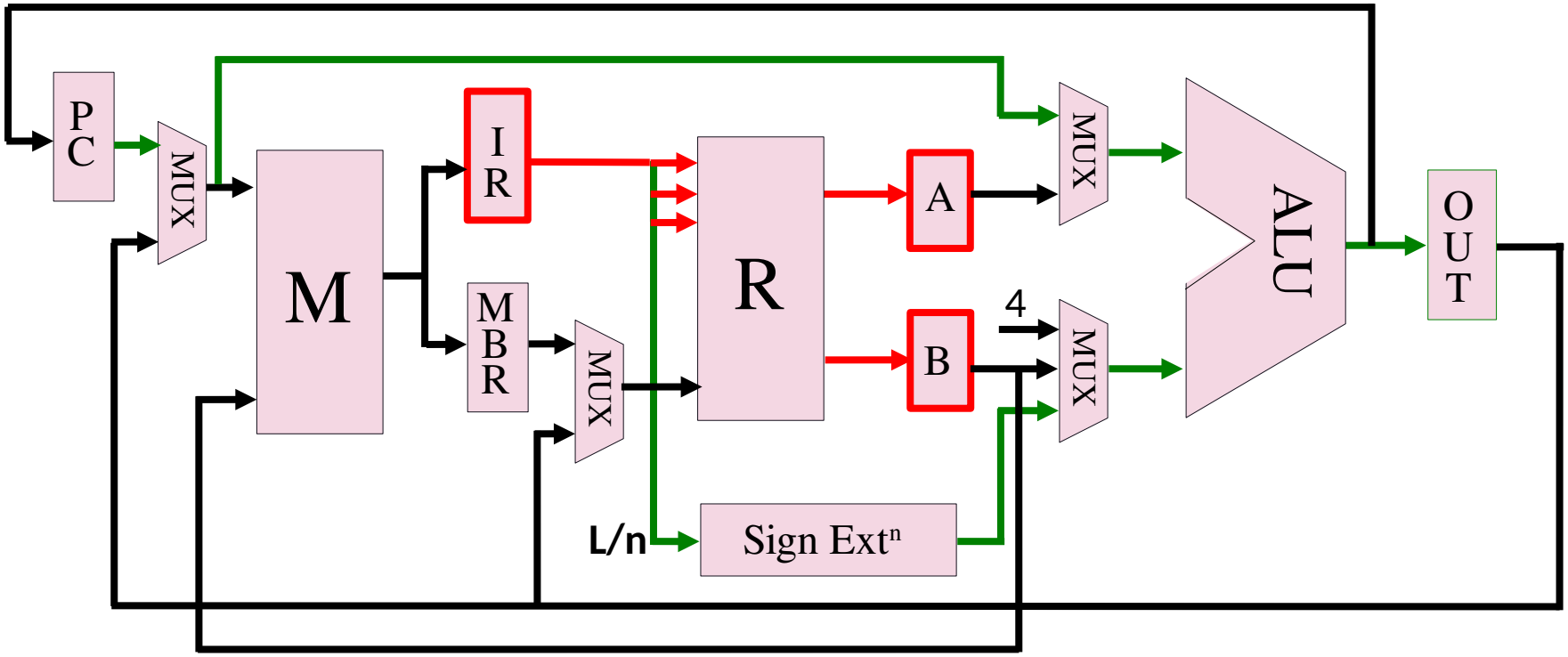


Optimized Design of CPU

Instruction execution

T_0 **Fetch**
IR=M[PC]
PC=PC+4

T_1 **ID & R_{RD}**
A= R[IR]
B= R[IR]
ALU_{out}= PC+ L



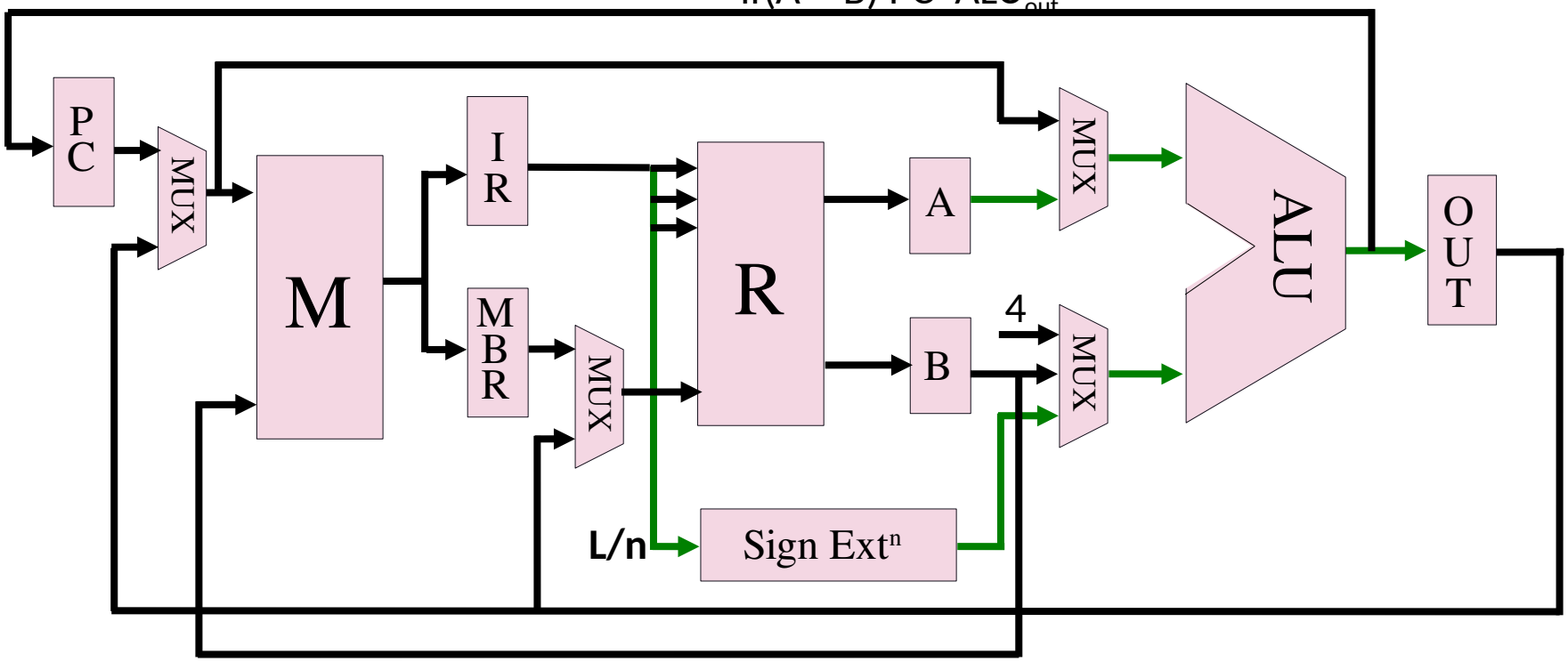
Optimized Design of CPU

Instruction execution

T_0 **Fetch**
IR=M[PC]
PC=PC+4

T_1 **ID & R_{RD}**
A= R[IR]
B= R[IR]
ALU_{out}= PC+ L

T_2 **Ex /M_{RD}/JMP**
ALU_{out}= A+n[IR]
ALU_{out}= f(A,B)
if(A==B) PC=ALU_{out}



add R_1, R_2, R_3 **sb R_1, n(R_2)** **lb R_1, n(R_2)** **jeq R_1, R_2, L**

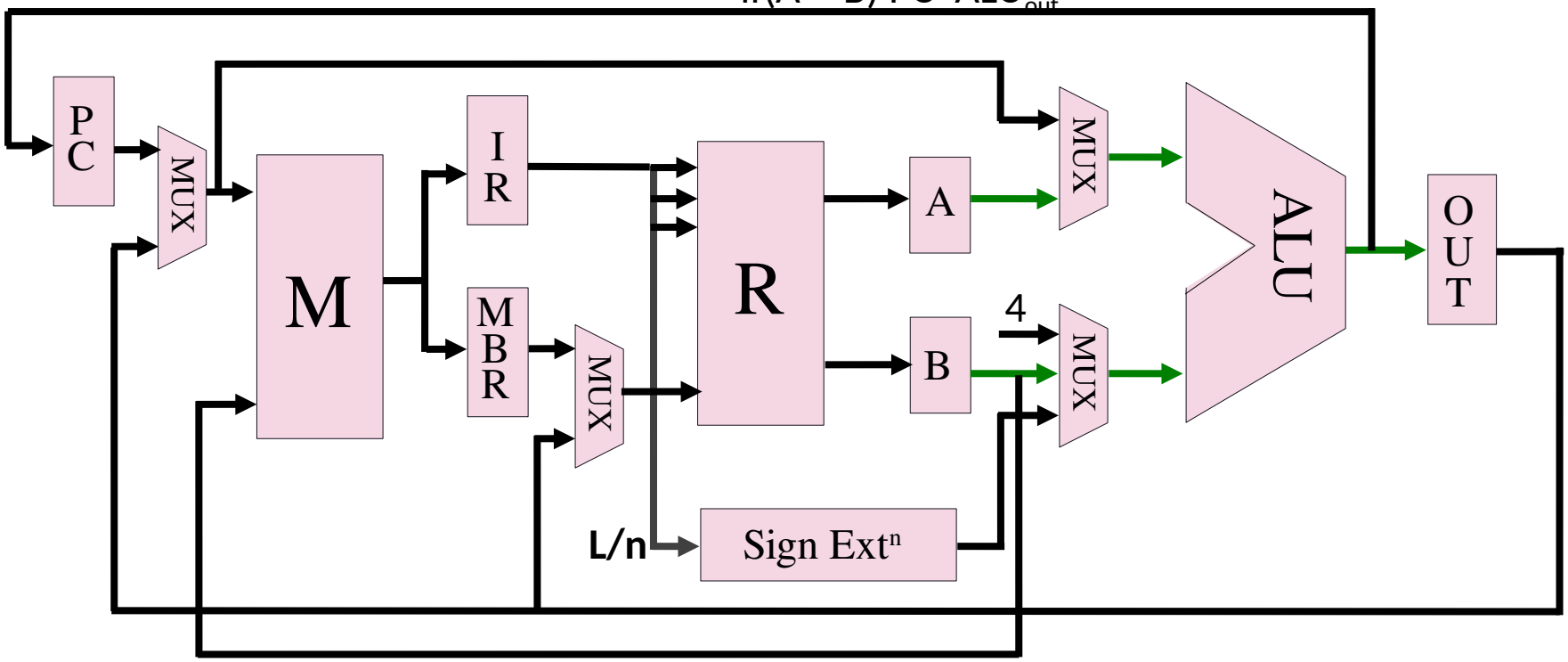
Optimized Design of CPU

Instruction execution

T_0 **Fetch**
IR=M[PC]
PC=PC+4

T_1 **ID & R_{RD}**
A= R[IR]
B= R[IR]
ALU_{out}= PC+ L

T_2 **Ex /M_{RD}/JMP**
ALU_{out}= A+n[IR]
ALU_{out}= f(A,B)
if(A==B) PC=ALU_{out}



add R_1, R_2, R_3 **sb R_1, n(R_2)** **lb R_1, n(R_2)** **jeq R_1, R_2, L**

Optimized Design of CPU

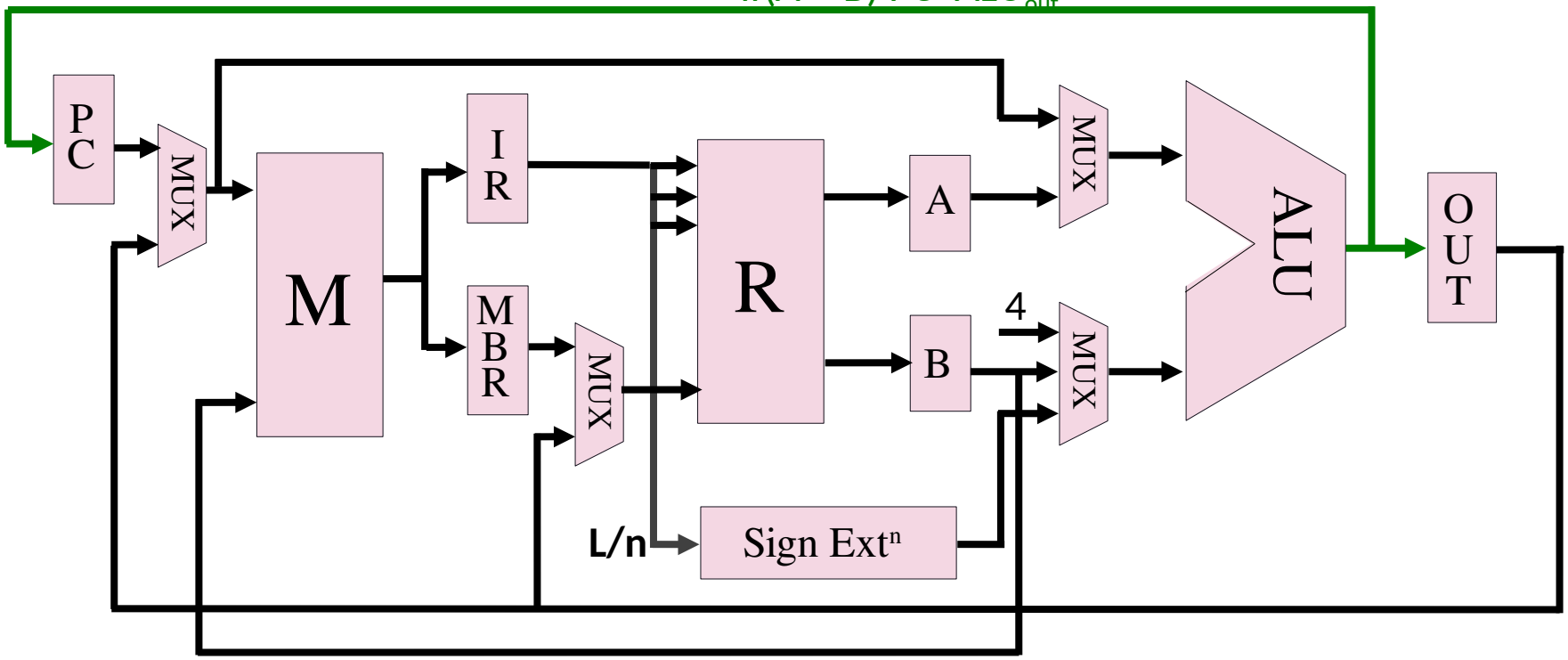
Instruction execution

T_0 **Fetch**
IR=M[PC]
PC=PC+4

T_1 **ID & R_{RD}**
A= R[IR]
B= R[IR]
ALU_{out}= PC+ L

T_2 **Ex /M_{RD}/JMP**
ALU_{out}= A+n[IR]
ALU_{out}= f(A,B)
if(A==B) PC=ALU_{out}

T_3 **Write back**
MBR=M[ALU_{out}] : LW
M[ALU_{out}] = B : ST
R[IR]=ALU_{out} : R



add R_1, R_2, R_3 **sb R_1, n(R_2)** **lb R_1, n(R_2)** **jeq R_1, R_2, L**

Optimized Design of CPU

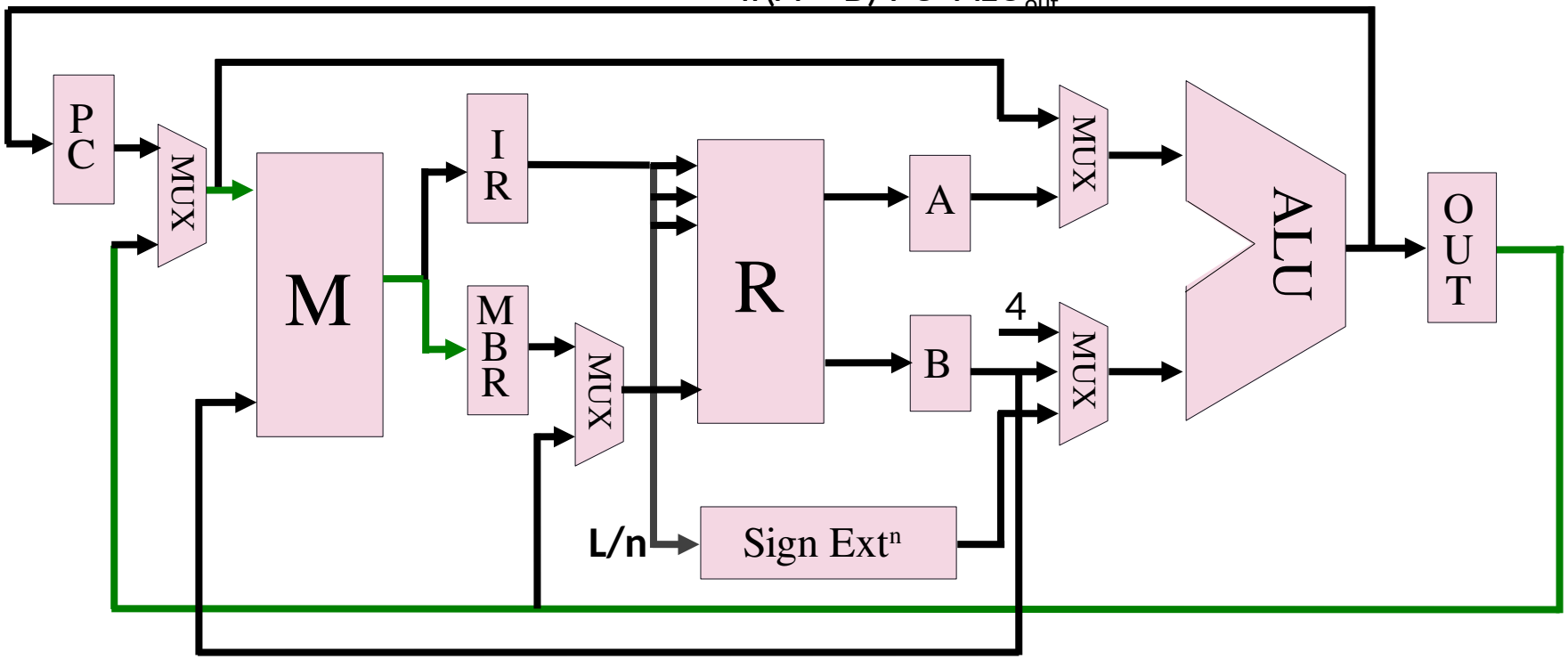
Instruction execution

T_0 **Fetch**
IR=M[PC]
PC=PC+4

T_1 **ID & R_{RD}**
A= R[IR]
B= R[IR]
ALU_{out}= PC+ L

T_2 **Ex /M_{RD}/JMP**
ALU_{out}= A+n[IR]
ALU_{out}= f(A,B)
if(A==B) PC=ALU_{out}

T_3 **Write back**
MBR=M[ALU_{out}] : LW
M[ALU_{out}] = B : ST
R[IR]=ALU_{out} : R



add R_1, R_2, R_3 **sb R_1, n(R_2)** **lb R_1, n(R_2)** **jeq R_1, R_2, L**

Optimized Design of CPU

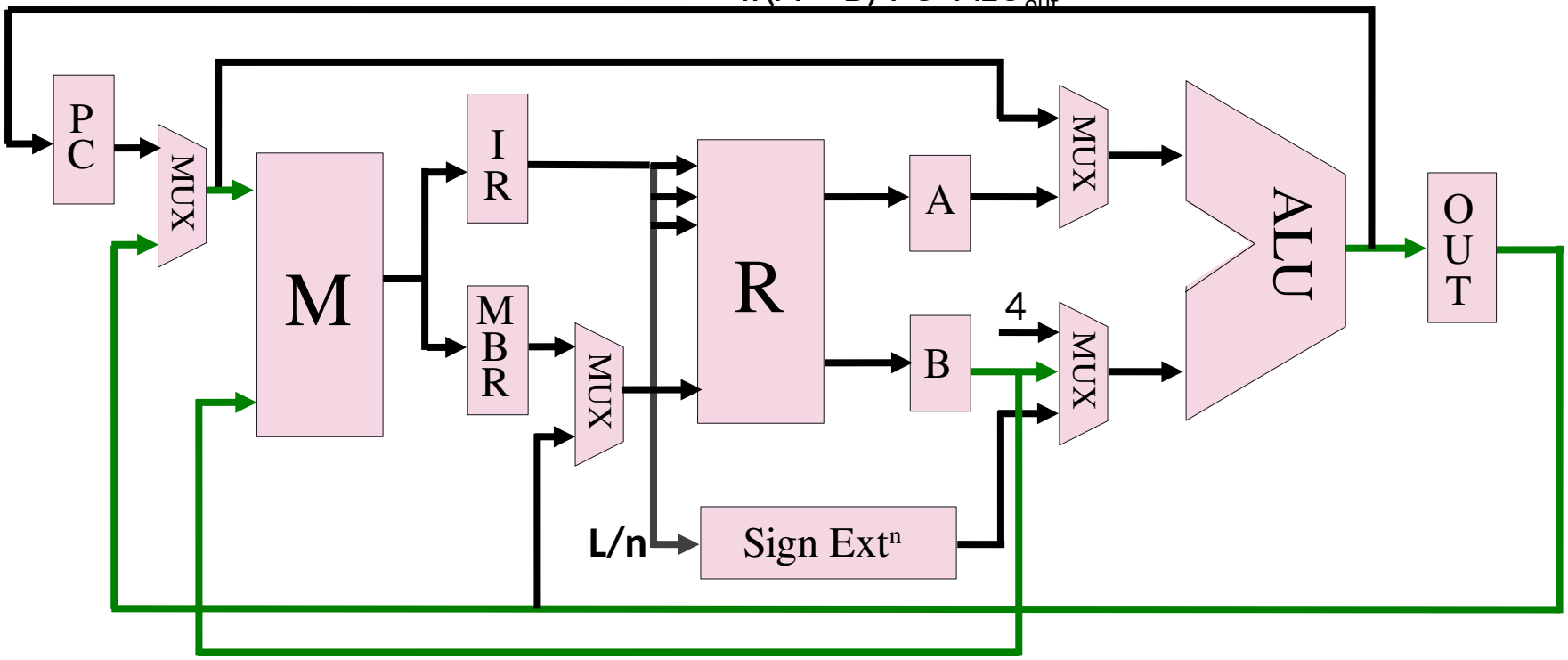
Instruction execution

T_0 **Fetch**
IR=M[PC]
PC=PC+4

T_1 **ID & R_{RD}**
A= R[IR]
B= R[IR]
ALU_{out}= PC+ L

T_2 **Ex /M_{RD}/JMP**
ALU_{out}= A+n[IR]
ALU_{out}= f(A,B)
if(A==B) PC=ALU_{out}

T_3 **Write back**
MBR=M[ALU_{out}] : LW
M[ALU_{out}] = B : ST
R[IR]=ALU_{out} : R



add R_1, R_2, R_3 **sb R_1, n(R_2)** **lb R_1, n(R_2)** **jeq R_1, R_2, L**

Optimized Design of CPU

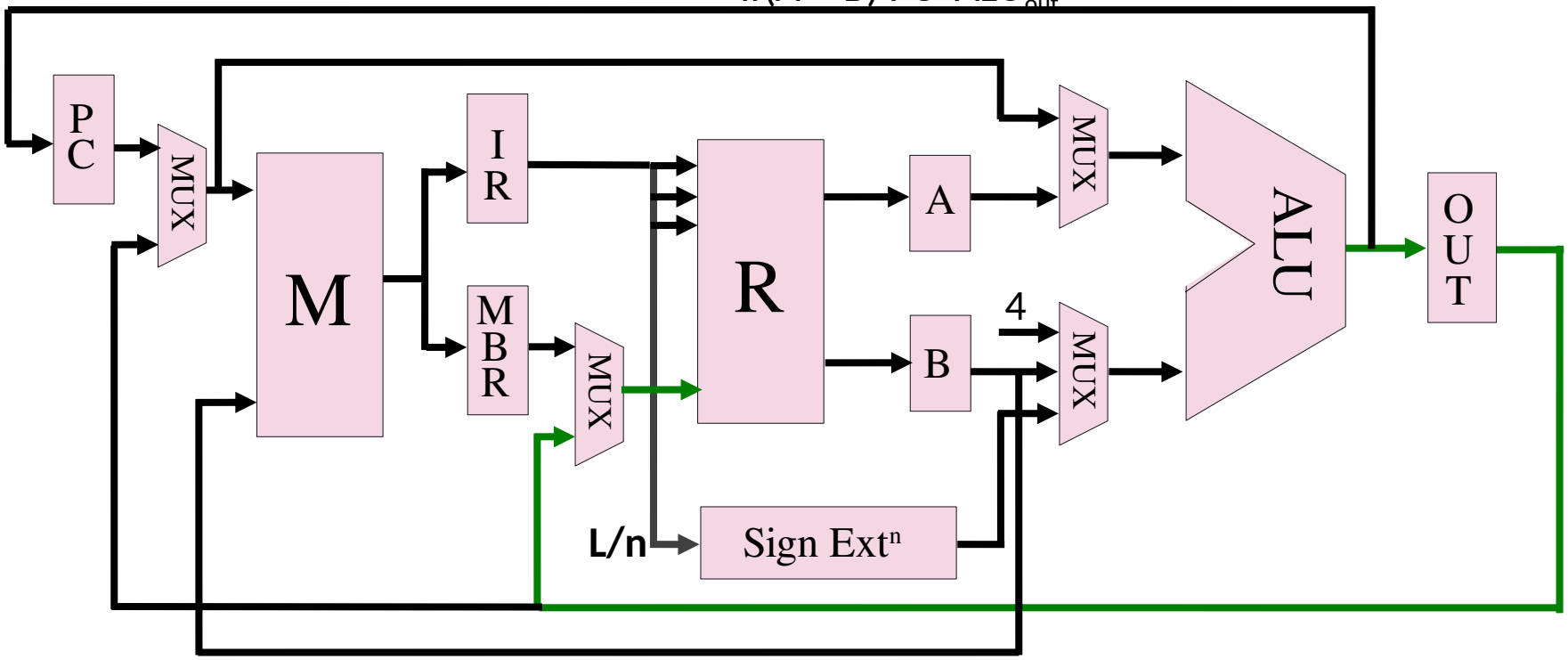
Instruction execution

T_0 **Fetch**
IR=M[PC]
PC=PC+4

T_1 **ID & R_{RD}**
A= R[IR]
B= R[IR]
ALU_{out}= PC+ L

T_2 **Ex /M_{RD}/JMP**
ALU_{out}= A+n[IR]
ALU_{out}= f(A,B)
if(A==B) PC=ALU_{out}

T_3 **Write back**
MBR=M[ALU_{out}] : LW
M[ALU_{out}] = B : ST
R[IR]=ALU_{out} : R



add R_1, R_2, R_3 **sb R_1, n(R_2)** **lb R_1, n(R_2)** **jeq R_1, R_2, L**

Optimized Design of CPU

Instruction execution

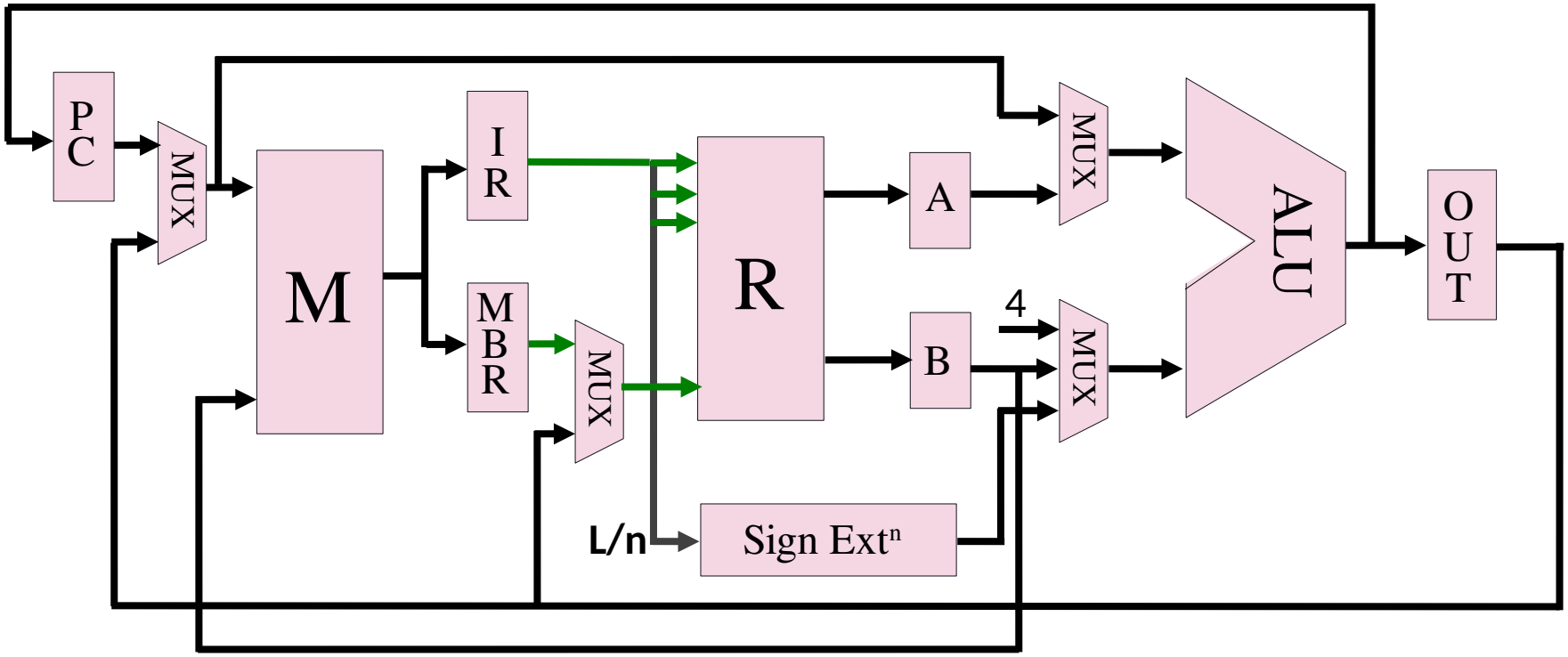
T_5 **LD**

R[IR]=MDR

Clock speed is faster

Hardware cost reduced

Pipelining is difficult



add R_1, R_2, R_3 **sb R_1, n(R_2)** **lb R_1, n(R_2)** **jeq R_1, R_2, L**

Optimized Design of CPU

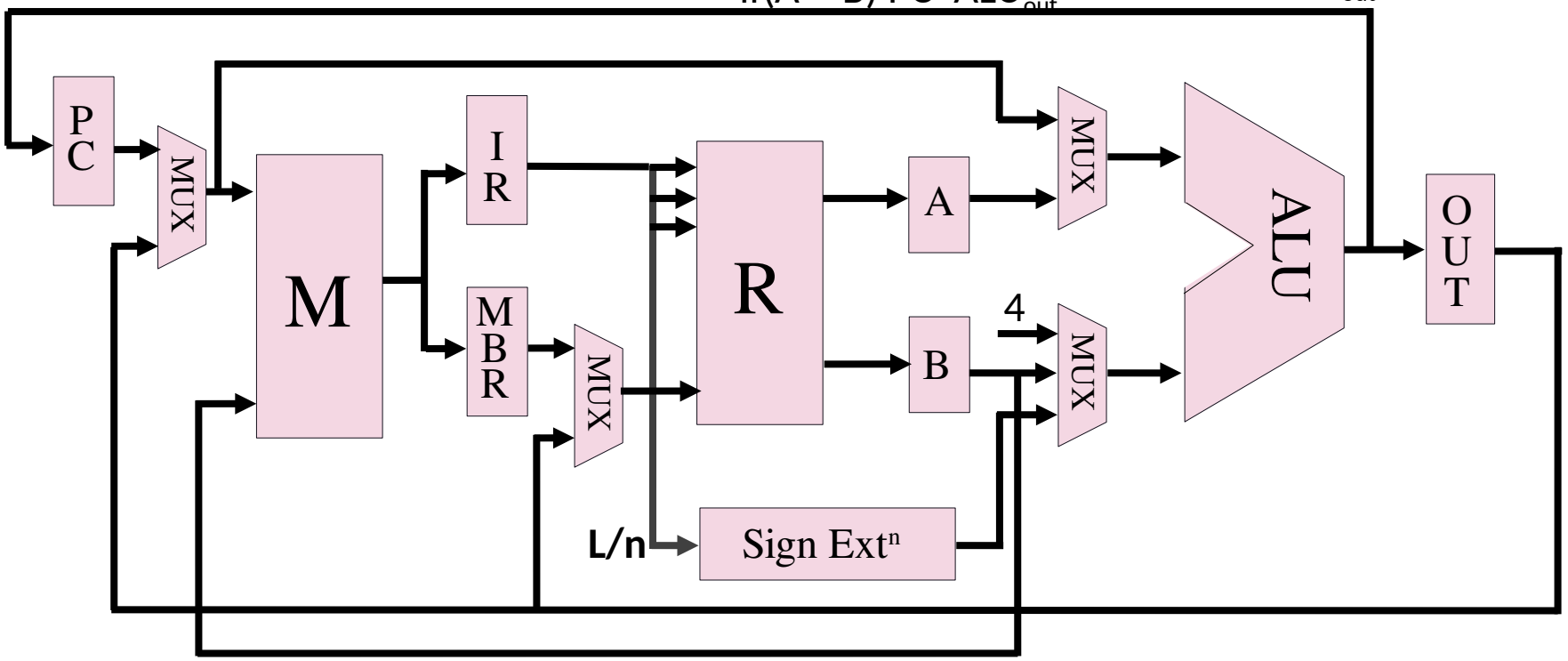
Instruction execution

T_0 **Fetch**
IR=M[PC]
PC=PC+4

T_1 **ID & R_{RD}**
A= R[IR]
B= R[IR]
ALU_{out}= PC+ L

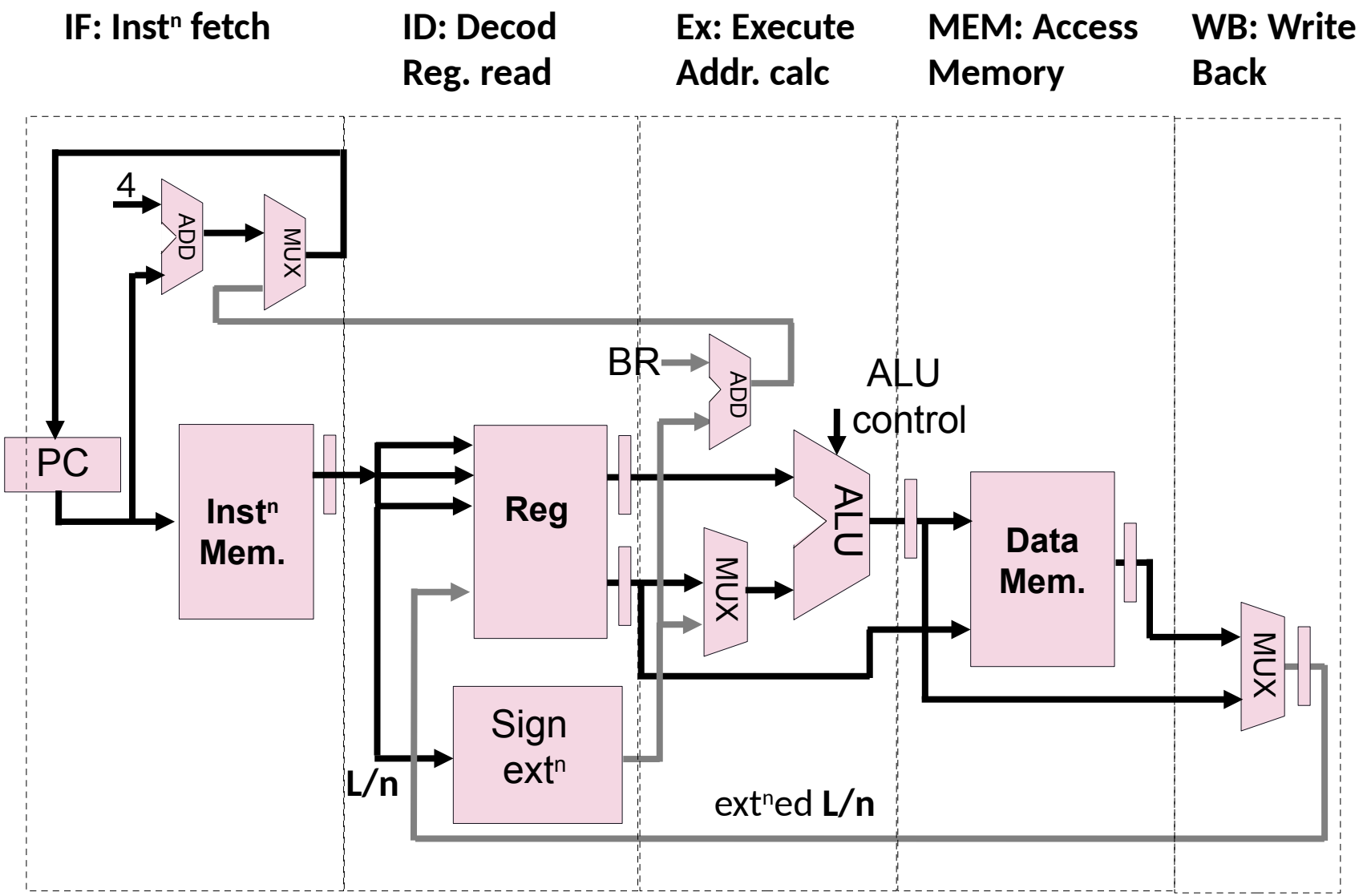
T_2 **Ex /M_{RD}/JMP**
ALU_{out}= A+n[IR]
ALU_{out}= f(A,B)
if(A==B) PC=ALU_{out}

T_3 **Write back**
MBR=M[ALU_{out}] : LW
M[ALU_{out}] = B : ST
R[IR]=ALU_{out} : MDR

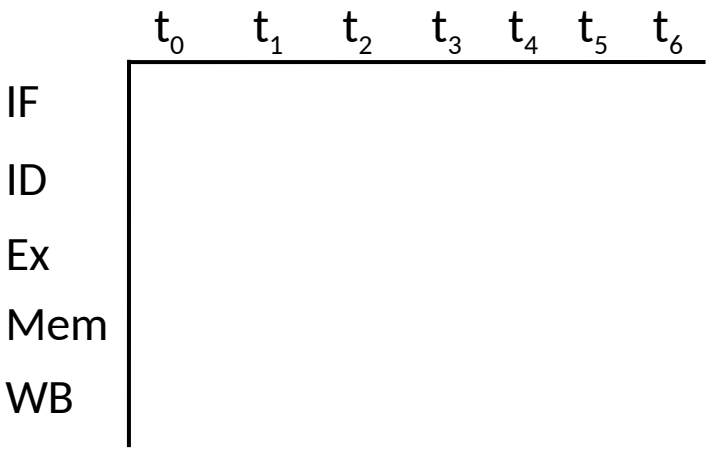


add R_1, R_2, R_3 **sb R_1, n(R_2)** **lb R_1, n(R_2)** **jeq R_1, R_2, L**

CPU Pipe-lined Design



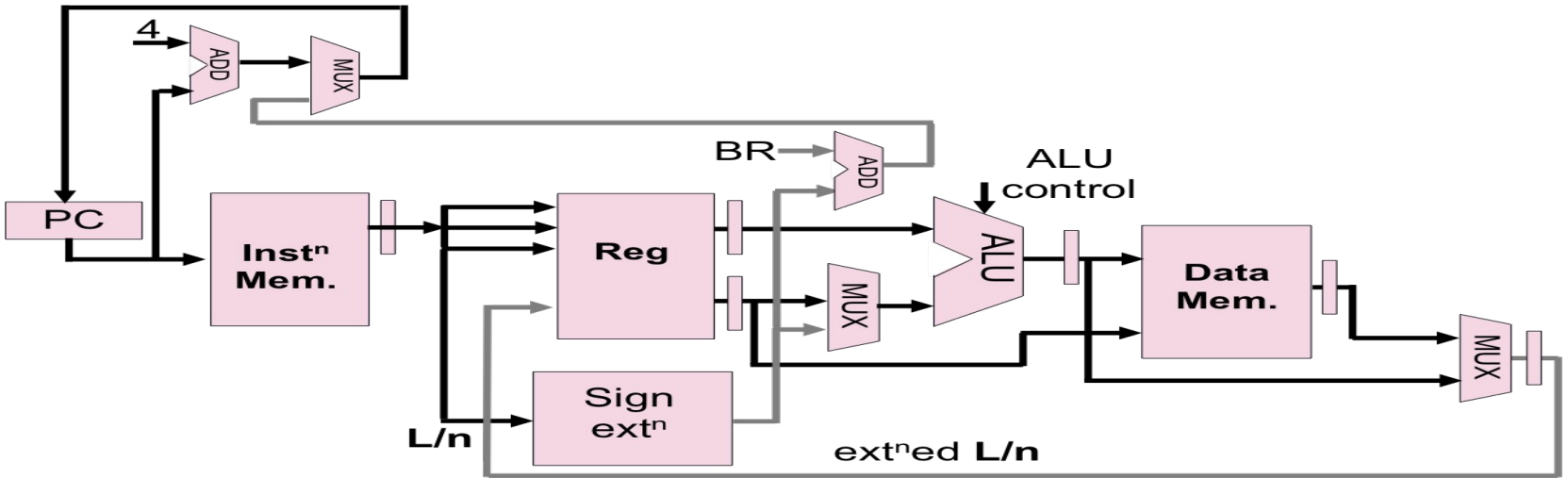
CPU Pipelining



Instruction Pipelining

1. *LD C, L*

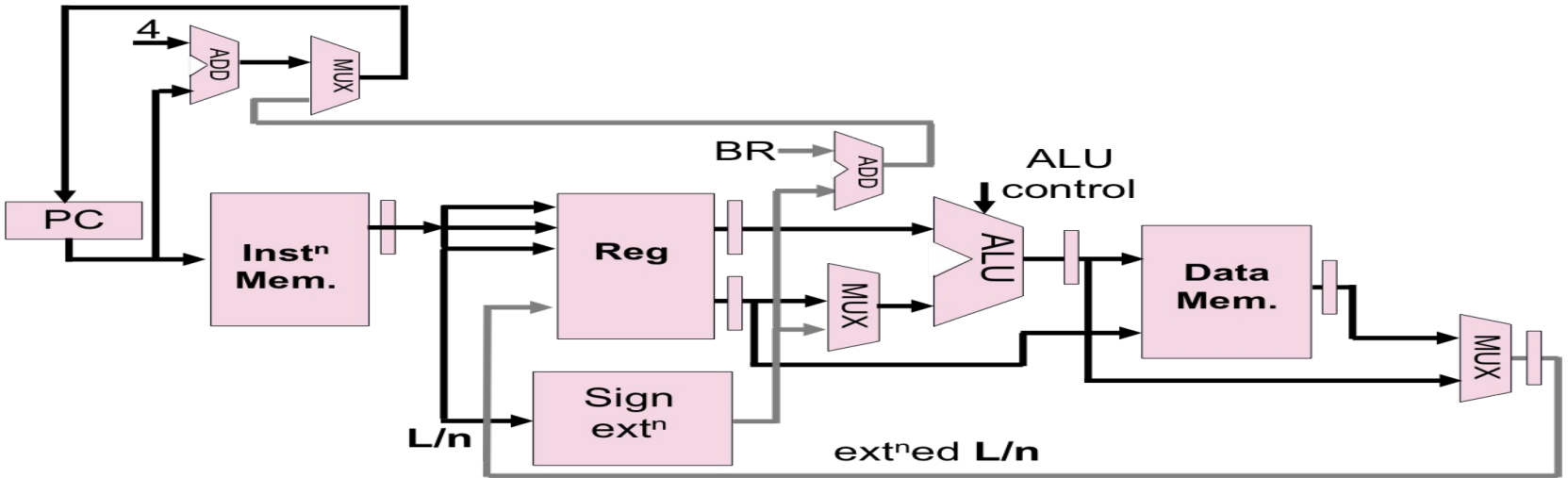
	t_0	t_1	t_2	t_3	t_4	t_5	t_6
IF	X						
ID							
Ex							
Mem							
WB							



Instruction Pipelining

1. *LD C, L*
2. *MOV B, C*

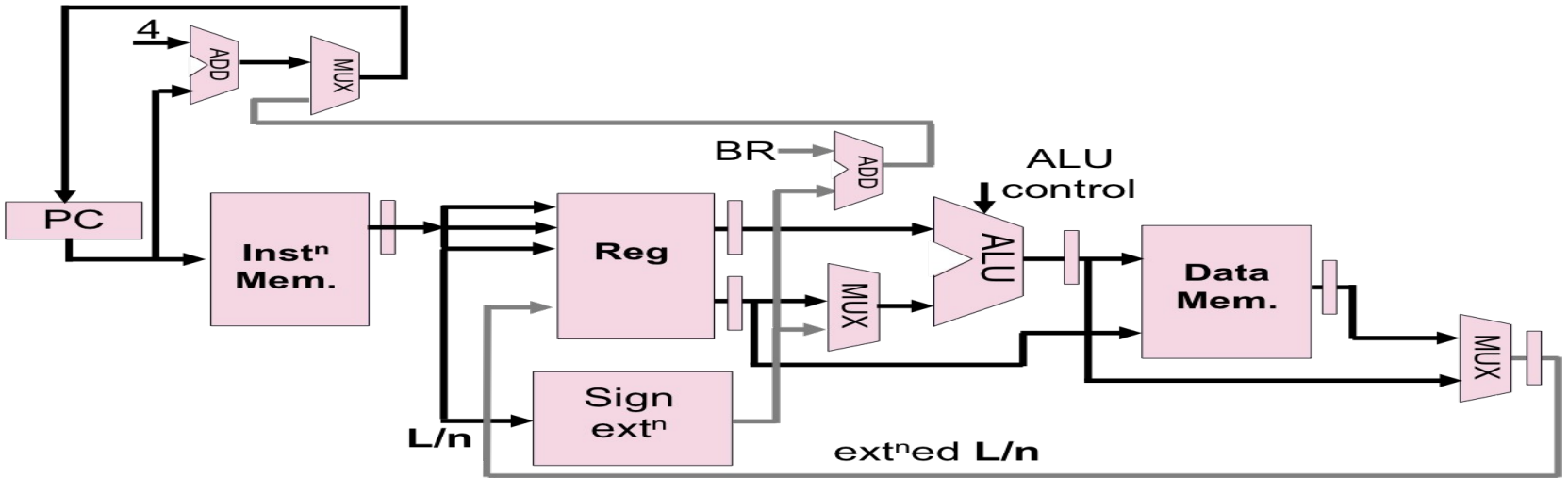
	t_0	t_1	t_2	t_3	t_4	t_5	t_6
IF	X	X					
ID		X					
Ex							
Mem							
WB							



Instruction Pipelining

1. *LD C, L*
2. *MOV B, C*
3. *CMP B*

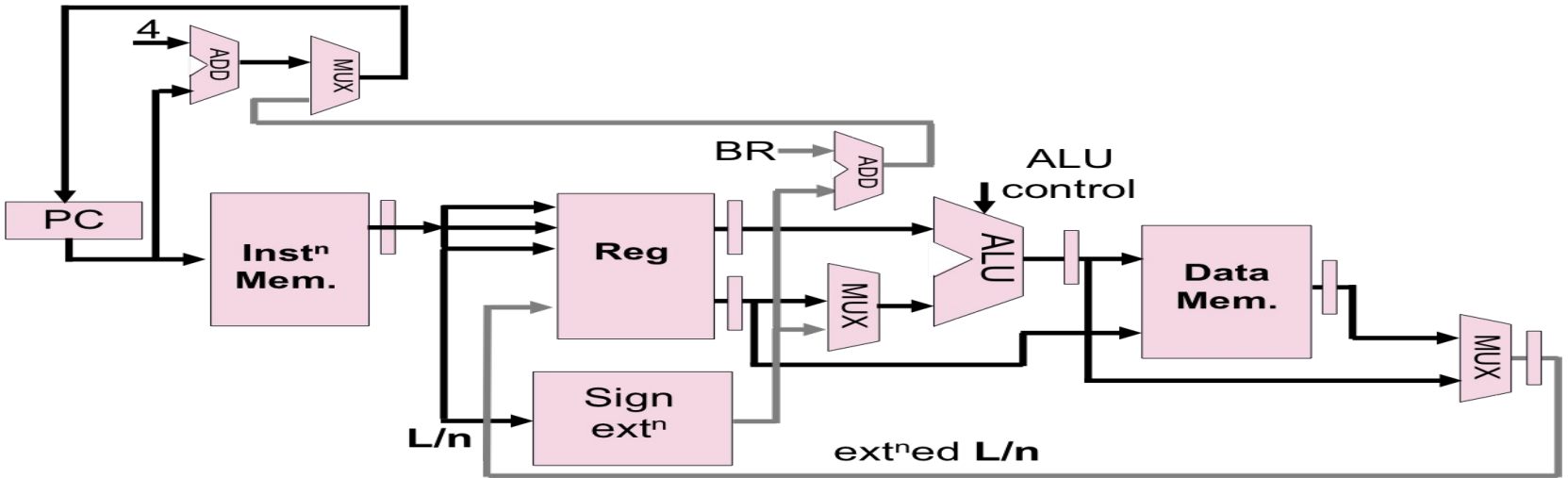
	t_0	t_1	t_2	t_3	t_4	t_5	t_6
IF	X	X	X				
ID		X	X				
Ex			X				
Mem							
WB							



Instruction Pipelining

1. *LD C, L*
2. *MOV B, C*
3. *CMP B*
4. *Add A, B,C*

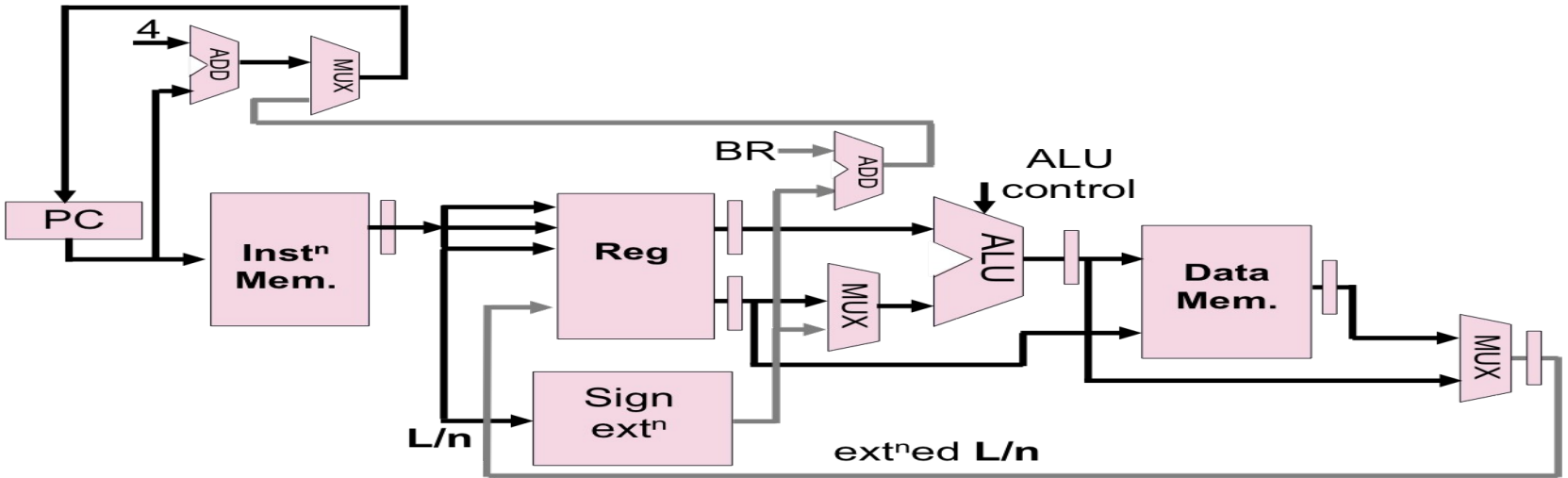
	t_0	t_1	t_2	t_3	t_4	t_5	t_6
IF	X	X	X	X			
ID		X	X	X			
Ex			X	X			
Mem				X			
WB							



Instruction Pipelining

1. *LD C, L*
2. *MOV B, C*
3. *CMP B*
4. *Add A, B,C*
5. *ST A, L*

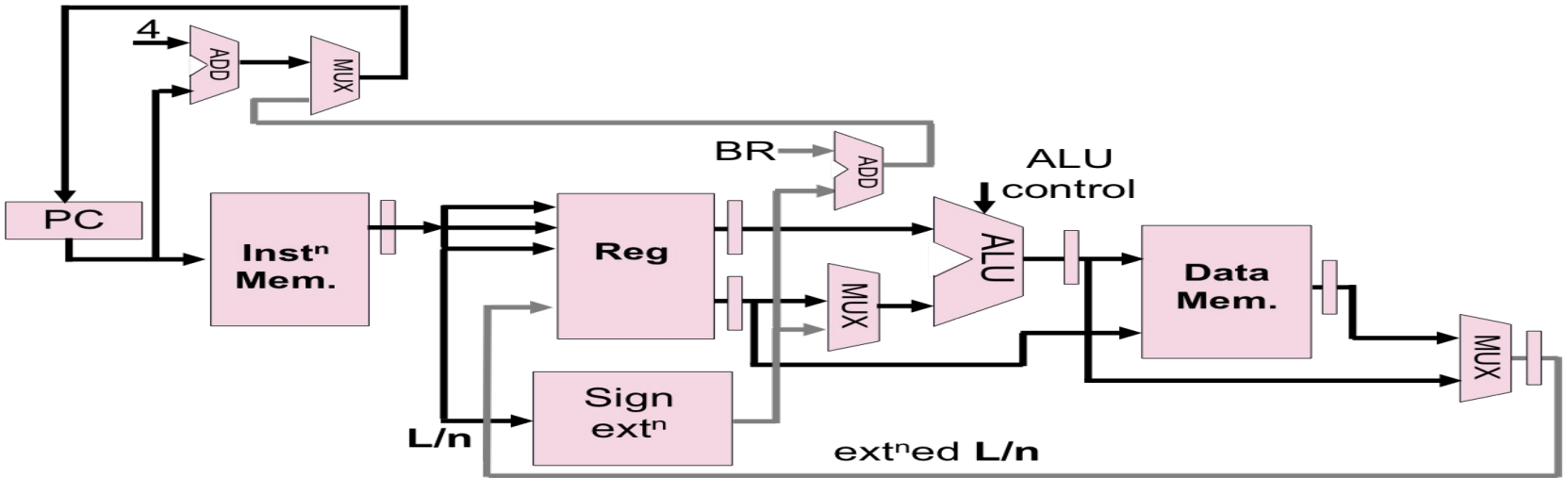
	t_0	t_1	t_2	t_3	t_4	t_5	t_6
IF	X	X	X	X	X		
ID		X	X	X	X		
Ex			X	X	X		
Mem				X	X		
WB					X		



Instruction Pipelining

1. *LD C, L*
2. *MOV B, C*
3. *CMP B*
4. *Add A, B,C*
5. *ST A, L*
6. *AND A,B, C*

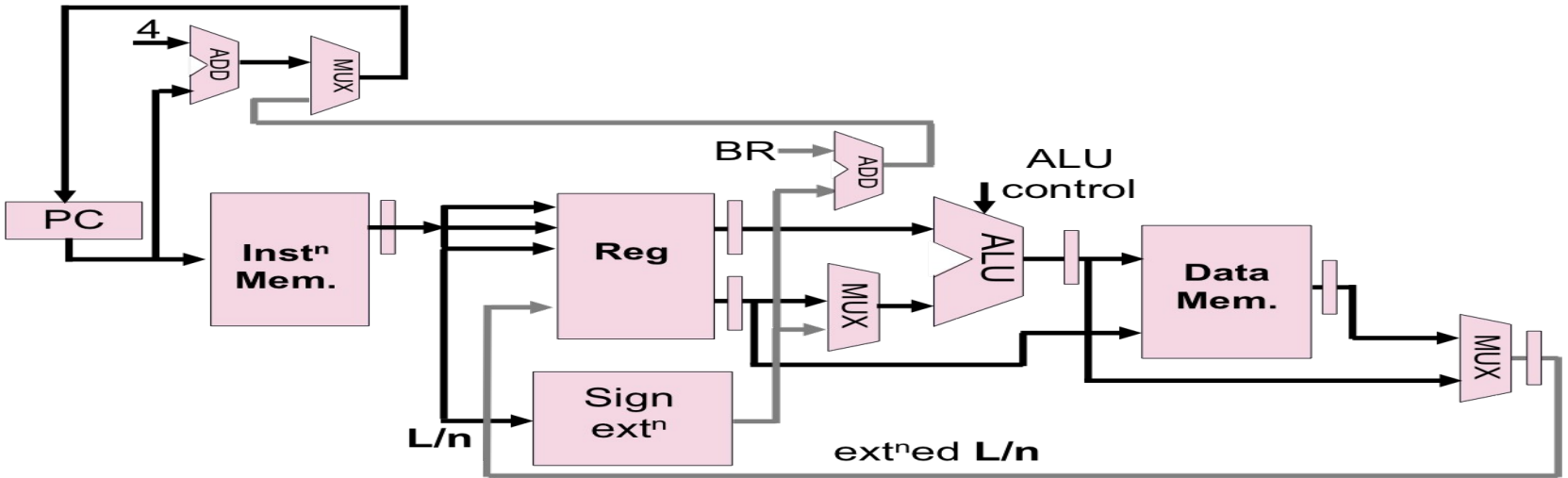
	t_0	t_1	t_2	t_3	t_4	t_5	t_6
IF	X	X	X	X	X	X	
ID		X	X	X	X	X	
Ex			X	X	X	X	
Mem				X	X	X	
WB					X	X	



Instruction Pipelining

1. *LD C, L*
2. *MOV B, C*
3. *CMP B*
4. *Add A, B,C*
5. *ST A, L*
6. *AND A,B, C*
7. *CMP A*

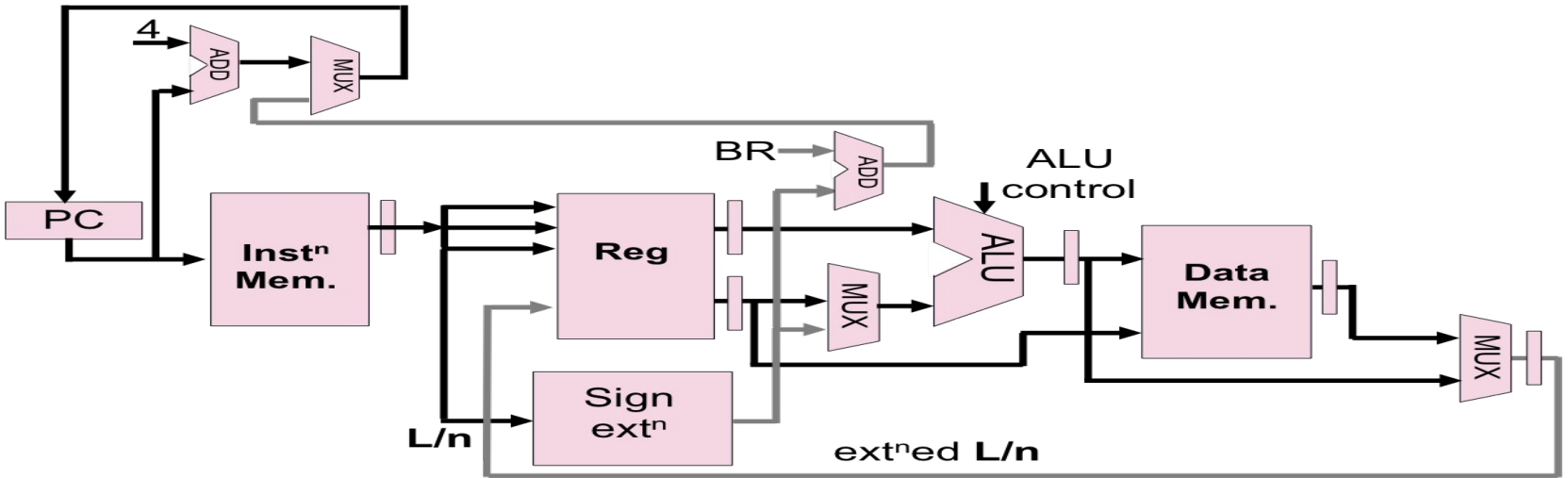
	t_0	t_1	t_2	t_3	t_4	t_5	t_6
IF	X	X	X	X	X	X	X
ID		X	X	X	X	X	X
Ex			X	X	X	X	X
Mem				X	X	X	X
WB					X	X	X



Instruction Pipelining

1. *LD C, L*
2. *MOV B, C*
3. *CMP B*
4. *Add A, B,C*
5. *ST A, L*
6. *AND A,B, C*
7. *CMP A*

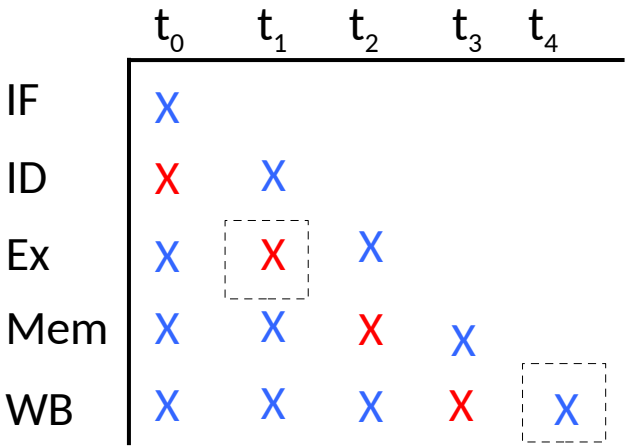
	t_0	t_1	t_2	t_3	t_4	t_5	t_6
IF	X	X	X	X	X	X	X
ID		X	X	X	X	X	X
Ex			X	X	X	X	X
Mem				X	X	X	X
WB					X	X	X



CPU Pipelining Hazards

Data hazard: One instruction input depend on the other instruction

Ex. Read and Write *Add A, B,C* *Add D, D,A*

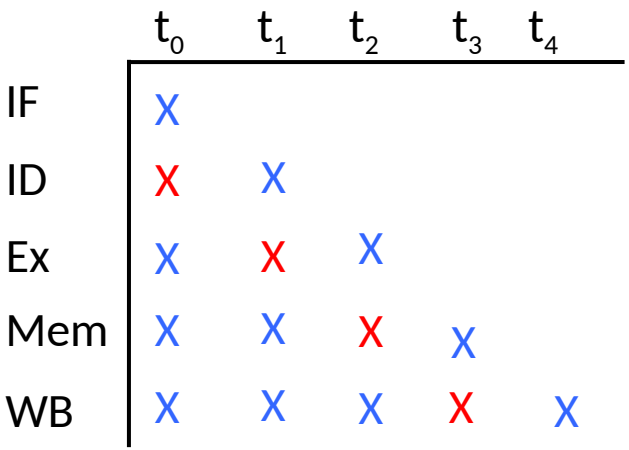


Data in A will be available in t4

CPU Pipelining Hazards

Control hazard: Decision of one instruction execution depend on other

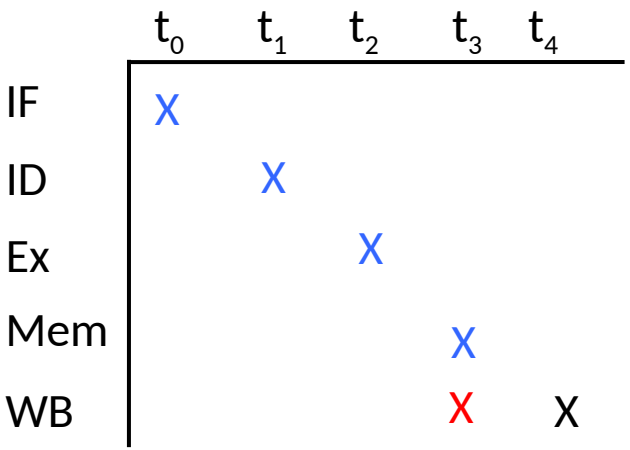
Ex. Branching *JEQ A, B,C*



CPU Pipelining Hazards

Control hazard: One execution depends on other

Ex. Single memory *ST A, L* *Add A, B,C*



Instruction Pipelining

1. *LD C, L*
2. *MOV B, C*
3. *CMP B*
4. *Add A, B,C*
5. *ST A, L*
6. *AND A,B, C*
7. *CMP A*

	t_0	t_1	t_2	t_3	t_4		
IF	X	X	X	X	X	X	X
ID		X	X	X	X	X	X
Ex			X	X	X	X	X
Mem				X	X	X	X
WB					X	X	X

